Gestión documental (producto sobre Multiplika MCP)

Idea

No es un almacén de documentos. Es un pipeline donde la IA lee cualquier documento de un cliente (PDF, foto, Word, etc.), lo interpreta y lo convierte a markdown estructurado, y ese markdown se indexa como base de conocimiento digitalizada de la empresa cliente — para que el cliente IA (Claude Desktop/Code conectado al MCP del proyecto) pueda consultarla y montar respuestas a las preguntas del usuario.

Objetivo: producto replicable — se monta este proyecto de gestión documental por separado a cada cliente que lo pida.

Sin interfaz propia: todo el flujo (subir documentos, corregirlos, consultarlos) pasa por conversación con el cliente IA vía tools MCP — no hay panel/formulario de gestión documental.

Decisiones de arquitectura

Multi-proyecto en un único servidor (2026-09-06)

Un único servidor/proceso MCP multi-proyecto, NO una instancia independiente por cliente. Se evita duplicar código en múltiples carpetas/despliegues.

- El aislamiento entre clientes se apoya en el mecanismo que ya tiene el Multiplika MCP: autenticación y contexto por proyecto (`obtener_contexto_proyecto` / `actualizar_contexto_proyecto` ya devuelven/actualizan datos scoped al proyecto autenticado).

Almacenamiento: Postgres + pgvector, 1 BD por cliente (2026-09-06, revisado)

- Una base de datos Postgres independiente por cliente (no un esquema compartido con `project_id`), por aislamiento fuerte y portabilidad (dar de baja/exportar un cliente de forma autocontenida).
- **`pgvector` se instala siempre** en la base de cada cliente (la extensión), da igual el modo de búsqueda que tenga activo — no tiene coste tenerla instalada sin usar.
- Se pasó de MariaDB a Postgres por la portabilidad que exige el esquema de 1 BD por cliente.
- Librería de acceso a datos: el usuario ya usa `mtpk-mariadb` (ORM propio en PyPI) en otros proyectos. Decisión: no tocar `mtpk-mariadb`; construir un paquete nuevo y separado, **`mtpk-postgres`**, con la misma interfaz/filosofía pero sobre `psycopg`. Repo local: `/home/jjandres/desarrollos/python3/mtpk_postgres`.
  - **Estado: publicado en PyPI como `mtpk-postgres` v0.1.1.** El bug que impedía crear/migrar tablas con índice ya está corregido y verificado — ver [[mtpk-postgres-adaptacion]].

Modo de búsqueda por cliente: fulltext o semántico (2026-09-06)

- Cada cliente tiene un modo de búsqueda configurable, guardado en su propia base (`gd_configuracion`): `fulltext` (búsqueda por palabras, nativa de Postgres, sin coste ni modelo de IA) o `semantico` (búsqueda por significado, vía embeddings + `pgvector`).
- **Cambiable en cualquier momento**, no fijo al alta. Por eso el esquema (índice de texto + tabla de embeddings) existe siempre en todos los clientes desde el principio — lo único condicionado por el modo activo es si el pipeline de ingesta genera o no el embedding de cada documento nuevo (eso sí

tiene coste real: llamada a la API). Si un cliente pasa de `fulltext` a `semantico` más adelante, hace falta un proceso de "backfill" que genere a posteriori los embeddings de los documentos ya ingeridos (pendiente de detallar cuando toque implementarlo).
- Un futuro modo "híbrido" (combinar las dos búsquedas) queda como posible mejora, no bloquea nada: con este diseño, añadirlo no exige tocar el esquema, solo lógica de consulta nueva.
- **Proveedor de embedding**: OpenAI `text-embedding-3-small` (1536 dimensiones) como estándar de la plataforma. La dimensión se guarda como parámetro por cliente en `gd_configuracion`, así que un cliente concreto podría usar otro proveedor en el futuro sin rediseñar el esquema.
- **Chunking**: por encabezados markdown (cada sección, con tope de tamaño), no por documento completo — necesario para que el embedding de un documento largo sea preciso.

Esquema de datos (2026-09-06)

Por cada base de datos de cliente:

```
gd_configuracion (1 fila)
  - modo_busqueda        TEXT ('fulltext' | 'semantico')
  - proveedor_embedding  TEXT (p.ej. 'openai:text-embedding-3-small')
  - dimension_embedding  INT  (1536 por defecto)

gd_documentos
  - id
  - nombre / referencia
  - tipo_documento       VARCHAR(50), texto libre (ver "Tipos de documento"
más abajo)
  - todas_versiones_buscables  BOOLEAN DEFAULT FALSE
  - fecha_alta

gd_tipos_documento (catálogo abierto de sugerencias, no una lista cerrada —
ver más abajo)
  - id
  - nombre               VARCHAR(50) UNIQUE
  - formato_requerido    TEXT  -- opcional, ver "Cumplimiento de
formato/normativa"
  - normativa_aplicable  TEXT  -- opcional, ídem
  - fecha_alta

gd_versiones
  - id
  - id_documento          FK
  - vigente               BOOLEAN   -- true solo en la última
  - borrador              BOOLEAN DEFAULT FALSE  -- ver "Borrador" más
abajo
  - markdown_original     TEXT      -- lo que generó la IA
  - markdown_actual       TEXT      -- vigente (= original o corregido a
mano)
  - tmp                   TEXT      -- borrador de edición del panel sin
confirmar, ver "Edición desde el panel"
  - tmp_id_usuario        INT       -- quién dejó ese borrador (→
mcpc_usuarios.id, sin FK real)
  - tmp_fecha             TIMESTAMP -- cuándo se escribió tmp por última
```

```
vez
  - revisado_manualmente  BOOLEAN DEFAULT FALSE
  - incumplimientos       TEXT  -- opcional, ver "Cumplimiento de
formato/normativa"
  - tsvector_actual       (GENERATED ALWAYS AS to_tsvector(markdown_actual)
STORED + índice GIN)
  - fecha_documento       -- la del propio documento, no la de subida
  - fecha_subida

gd_historial_ediciones (ver "Edición desde el panel")
  - id
  - id_version         FK
  - id_usuario         INT (→ mcpc_usuarios.id, sin FK real)
  - markdown_anterior  TEXT      -- snapshot completo de markdown_actual
justo antes de esta confirmación
  - fecha

gd_chunks (solo si el modo es/fue semántico, y solo de versiones buscables)
  - id
  - id_version    FK
  - orden
  - texto
  - embedding     VECTOR(dimension_embedding) + índice HNSW
```

Búsqueda (full-text o semántica): siempre filtra por `vigente = true`, salvo que `gd_documentos.todas_versiones_buscables = true` (entonces entran todas las versiones de ese documento).

Correcciones — dos escenarios contemplados:

- **A) Nueva versión del documento** (el cliente sube una versión corregida/actualizada): nueva fila en `gd_versiones`, la anterior deja de ser `vigente`. Se guarda todo el histórico, sin límite de tiempo ni purga (decisión: el espacio que ocupa es marginal — texto y vectores pequeños; no se almacena el binario original).
- **B) Corrección del markdown mal generado por la IA** (sin que haya documento nuevo del cliente): se edita `markdown_actual` de la versión existente, conservando `markdown_original` como lo que generó la IA la primera vez, y marcando `revisado_manualmente=true`. Mecanismo (decidido 2026-09-08): siempre vía tool MCP (`corregir_documento_gd`), a petición del propio cliente a su IA — sin revisión de nadie del equipo, porque no hay capacidad para tenerla. El riesgo asumido es que una corrección mal hecha por la IA del cliente quede guardada sin que nadie la verifique; se acepta porque `markdown_original` nunca se pierde.

Borrador (2026-09-13)

Problema: a veces llega un documento incompleto (le faltan páginas, el cliente aún lo está revisando) y las idas y vueltas hasta cerrarlo no deberían ensuciar el histórico de `gd_versiones` con una versión por cada corrección intermedia.

Solución: `gd_versiones.borrador` (BOOLEAN, default false). Mientras la versión vigente de un documento sea un borrador, `guardar_documento_gd(id_documento_existente=..., ...)`

sobrescribe esa misma fila (incluido `markdown_original` — única excepción a "nunca se sobrescribe", justificada porque mientras es borrador no es un documento real todavía) en vez de crear una versión nueva. El cliente lo da por bueno de dos formas: llamando otra vez a `guardar_documento_gd` con `borrador=false` (guarda el contenido final y lo cierra en el mismo paso), o con `confirmar_documento_gd` si el contenido no cambia y solo hace falta la confirmación. A partir de ahí, `borrador=false` para siempre en esa versión: cualquier guardado posterior sobre ese documento vuelve a crear una versión nueva, como con cualquier documento normal — el flujo A de arriba no cambia.

Por defecto `borrador=false`: un documento que llega completo se guarda igual que siempre, sin que el cliente tenga que saber que existe este mecanismo.

Edición desde el panel (2026-09-13)

Hasta ahora el panel del cliente era de solo lectura: guardar o corregir un documento era siempre por conversación con la IA. Se añade edición directa del texto desde el panel, con un control de cambios (quién y cuándo) y la posibilidad de revertir.

Rol nuevo: invitado. El catálogo de roles (`mcpc_roles` del core) gana un cuarto rol, `invitado` (id=4): acceso de solo lectura, no puede editar nada. `admin_tenant` y `operador` sí pueden editar. `lib/roles_proyecto.puede_editar_proyecto()` es la comprobación (cualquier rol salvo invitado); un superadmin NO puede editar documentos por este camino — el historial de ediciones referencia `mcpc_usuarios`, no `mcpc_superadmins`, así que para dar soporte editando algo hay que entrar con un usuario real de ese tenant.

Guardado sin ensuciar el historial: `gd_versiones.tmp`. Mientras se edita, cada "guardar" del panel escribe en `tmp` (+ `tmp_id_usuario`, `tmp_fecha`) sin tocar `markdown_actual` ni generar ningún registro — exactamente igual que guardar un archivo no es un commit en git. Solo una acción explícita, "confirmar", cierra la edición: el `markdown_actual` de antes pasa a `gd_historial_ediciones` y `tmp` pasa a ser el nuevo `markdown_actual` (y se vacía). Si el editor se reabre con `tmp` todavía con contenido, es que quedó una edición sin confirmar (de quien sea, no hay bloqueo entre usuarios) — se carga ese texto en vez del confirmado, con un aviso visible de quién lo dejó y cuándo, para que quien lo retome sepa lo que está haciendo.

Historial: snapshot completo, no solo el diff. `gd_historial_ediciones` guarda, en cada confirmación, el texto completo de `markdown_actual` de justo antes (columna `markdown_anterior`), no un diff — el coste en espacio es marginal (mismo razonamiento que ya justifica no purgar nunca `gd_versiones`). El diff que ve el cliente se calcula al vuelo (`difflib.unified_diff`, formato tipo `git diff`) comparando dos snapshots consecutivos, solo cuando alguien lo pide. Guardar el snapshot completo (y no solo el diff) es lo que hace trivial revertir: copiar `markdown_anterior` de vuelta a `markdown_actual` — y ese propio revertir queda registrado como una confirmación más, nunca se pierde el rastro de qué pasó.

Esto es independiente del mecanismo de "Borrador" de más arriba (que evita versiones nuevas mientras un documento entero está sin cerrar, vía chat): la edición desde el panel corrige el texto de la versión vigente, sea o no un borrador.

Alta manual desde el panel (2026-09-24)

Hasta ahora el panel solo dejaba editar un documento ya existente (ver arriba) o encolar un fichero crudo en `gd_documentos_pendientes` a la espera de que la IA lo procesara en conversación — no había forma de

dar de alta un documento nuevo sin pasar por la IA. Se añade `POST /v1/proyectos/{id}/documentos`: el propio usuario pega o escribe el markdown ya convertido (mismos metadatos que pide `guardar_documento_gd`: nombre, tipo, fecha, ámbito, caducidad, histórico buscable) y el panel llama directamente a `guardar_documento()` — mismo camino de guardado que usa la tool MCP, sin duplicar lógica de negocio.

Decisión de alcance (2026-09-24): sin OCR/conversión automática en el panel — convertir un PDF o una foto a markdown lo sigue haciendo solo la IA en conversación (es su punto fuerte, y montar un pipeline de extracción propio en el panel sería peor calidad y mucho más código). Lo que sí se añade es poder **adjuntar el PDF original tal cual** al crear a mano (`gd_versiones.archivo_original` + `_nombre` + `_tipo_mime`, BLOB opcional): queda guardado solo como referencia descargable junto al documento — `markdown_actual` sigue siendo lo único que se busca y lo que manda. Mismo permiso que editar (`puede_editar_documento_actual`, ver "Edición desde el panel"); si el modo de búsqueda del proyecto es semántico, se reindexa igual que si lo hubiera guardado la IA.

Pipeline de ingesta (2026-09-06)

Sin interfaz: el cliente le dice a su Claude "guárdame esta factura" (la pega o adjunta) y es el propio cliente IA quien gestiona todo el proceso vía tools MCP.

La IA **siempre pregunta** al usuario antes de guardar (no infiere del contenido), como mínimo:

1. Tipo de documento — consultando antes `listar_tipos_documento_gd()` para sugerir uno ya existente (ver "Tipos de documento" más abajo).
2. Nombre/referencia corta para identificarlo después.
3. Si es una versión nueva de un documento ya guardado o uno distinto — si dice que es nueva versión, la IA debe confirmar cuál usando `buscar_documentos()`, no adivinar.
4. Fecha del propio documento (no la de subida).
5. Si ese documento necesita que todo su histórico quede buscable, o solo la última versión (por defecto: solo la última).

Tools MCP (2026-09-06)

Tres herramientas cubren todo el ciclo de consulta (la de ingesta/guardado queda para cuando se aborde la implementación, junto con el mecanismo de corrección del punto B de arriba):

1. **`consultar_documentos(pregunta, limite=5)`** — la principal, para responder preguntas del cliente. Resuelve sola el modo de búsqueda del proyecto (fulltext o semántico) y devuelve los fragmentos más relevantes con su documento, tipo, fecha e identificadores, para que la IA cite la fuente y pueda pedir el documento completo si hace falta.
2. **`obtener_documento_completo(id_documento, version=None)`** — devuelve el markdown completo de la versión vigente (o la indicada), para cuando un fragmento no basta.
3. **`buscar_documentos(referencia)`** — búsqueda por metadatos (nombre, tipo, fecha), no por contenido; sirve tanto para consulta exploratoria como para evitar duplicados al ingerir.

`obtener_estado_proyecto_gd()` (añadida 2026-09-09): total de documentos guardados y fecha de la última alta (documento o versión nueva) — es el dato de estado por proyecto planteado en `docs/decisiones.md` del repo core, apartado "Contexto compartido: separar negocio de preferencias"

(paso 3). No refleja correcciones hechas con `corregir_documento_gd`, solo altas — para no necesitar tocar el esquema. Vive aquí porque es específico de este producto, no de la plataforma core.

`archivar_documento_gd(id_documento)` / `desarchivar_documento_gd(id_documento)` (añadidas 2026-09-09): un documento que ya no aplica (anulado, subido por error) se marca `archivado` en vez de borrarse — desaparece de `buscar_documentos_gd` (salvo `incluir_archivados=True`) y de `consultar_documentos_gd` (fulltext y semántico), pero sigue accesible por id y se puede recuperar sin perder nada del histórico.

`confirmar_documento_gd(id_documento, id_version=None)` (añadida 2026-09-13): cierra un borrador (ver "Borrador" más arriba) sin cambiar su contenido.

`listar_credenciales_gd()` (añadida 2026-09-09): nombre, descripción y usuario (si lo lleva) de cada credencial del proyecto — nunca la contraseña/API key. Ver "Credenciales" más abajo.

`listar_estilos_informe_gd()` / `obtener_estilo_informe_gd(nombre_tipo)` (añadidas 2026-09-09): tipos de informe disponibles, e instrucciones + imágenes fijas de uno en concreto (las imágenes llegan como contenido nativo de imagen de la propia respuesta, no como URL). Ver "Tabla de estilos" más abajo.

Tipos de documento: catálogo abierto, no lista cerrada (2026-09-10)

`tipo_documento` empezó siendo una lista cerrada de 5 valores fija en el esquema (factura/certificado/contrato/informe/otro). El cliente pidió que fuera "amplia, incluso configurable por el propio cliente" — se sustituyó por texto libre (`VARCHAR(50)`) más una tabla de sugerencias, `gd_tipos_documento`, que NO restringe qué se puede guardar: es solo una lista de "ya usados" para que la IA reutilice uno existente en vez de crear variaciones del mismo tipo (p.ej. "Factura" vs "facturas").

- **`listar_tipos_documento_gd()`** (tool MCP): devuelve el catálogo. La IA la consulta antes de preguntar el tipo al guardar un documento, y sugiere uno de los ya existentes si encaja.
- **`guardar_documento_gd`**: si el tipo indicado no está en el catálogo (y es un documento nuevo, no una versión nueva de uno existente), se añade solo — sin que el cliente tenga que darlo de alta antes en el panel.
- **Gestión desde el panel**: `GET/POST /v1/proyectos/{id}/tipos-documento` (cualquier usuario del proyecto) y `PUT`/`DELETE /v1/proyectos/{id}/tipos-documento/{id_tipo}` (solo admin) — un cliente puede curar su propio catálogo (añadir tipos por adelantado, borrar uno mal escrito, definir formato/normativa — ver "Cumplimiento de formato/normativa") sin depender de nosotros. Borrar una entrada del catálogo no afecta a los documentos que ya usan ese texto como `tipo_documento` — son independientes, el catálogo es solo una ayuda de sugerencia.
- Al aplicar el esquema (`update_db.py`) se siembran los 5 tipos por defecto **solo si la tabla está vacía** — así un cliente que ya haya editado su catálogo no los ve reaparecer en cada migración futura.

Gotcha de migración encontrado al probarlo (2026-09-10): el proyecto ya provisionado (`desarrollo_interno_gestion_documental_dev`) tenía, además del tipo de columna, una restricción `CHECK` residual (`gd_documentos_tipo_documento_check`, `tipo_documento IN ('factura', ...)`) de una versión anterior del esquema. `ManagerDB.crear_todo()` de `mtpk_postgres` no la detectó ni la quitó (ni siquiera con `--permitir-drop`, que solo cubre índices/FKs huérfanos) — guardar un tipo nuevo fallaba con un error de Postgres hasta borrar la restricción a mano (`ALTER TABLE gd_documentos DROP CONSTRAINT gd_documentos_tipo_documento_check`). Si en el futuro hay más clientes con

Postgres ya provisionados antes de este cambio, hay que revisar si tienen la misma restricción residual antes de darlos por migrados.

Cumplimiento de formato/normativa (2026-09-13)

Un tipo de documento puede llevar, opcionalmente, `formato_requerido` (cómo debe estar estructurado, p.ej. qué campos lleva una factura) y/o `normativa_aplicable` (qué reglamento debe cumplir). Es texto libre en ambos casos, no una validación estructurada — sigue sin haber lista cerrada ni esquema por tipo, esto es una guía para que la IA la lea y la aplique con criterio, no una regla mecánica.

- **Gestión**: `PUT /v1/proyectos/{id}/tipos-documento/{id_tipo}` (solo admin, página "Tipos de documento" del panel) para definir o borrar estos dos campos de un tipo. No afecta a documentos ya guardados.
- **Al guardar**: antes de `guardar_documento_gd`, la IA consulta `listar_tipos_documento_gd()` — si el tipo del documento tiene requisitos, compara el contenido contra ellos y describe en `incumplimientos` (parámetro de `guardar_documento_gd`) lo que no cumpla, en sus propias palabras. Si cumple, o el tipo no tiene requisitos, lo deja vacío.
- **Dónde queda**: `gd_versiones.incumplimientos` — por versión, no por documento (una versión corregida puede dejar de incumplir mientras otras versiones antiguas siguieran sin cumplir, aunque en la práctica solo importa la vigente). El panel lo muestra como aviso en rojo en la ficha del documento.
- **Resolución**: no se limpia solo. Cuando el documento se corrige (`corregir_documento_gd`, `guardar_documento_gd` con una versión nueva, o edición desde el panel) y la IA valora que ya cumple, llama a la tool nueva `actualizar_incumplimientos_gd(id_documento, incumplimientos=None)` para vaciarlo. Si sigue sin cumplir pero por otro motivo, actualiza el texto en vez de vaciarlo.

Credenciales (2026-09-09, ampliada 2026-09-10)

Gestor genérico de credenciales del cliente para conectar con servicios externos — sin un servicio concreto todavía, así que se diseñó abierto: un nombre, para qué sirve, un usuario opcional y la contraseña/API key/token, cifrada.

Primera versión (2026-09-09) metía usuario+contraseña como JSON dentro de un único campo `valor_cifrado` — se corrigió el mismo día siguiente porque escribir JSON a mano no es razonable para alguien no técnico: ahora `usuario` es su propia columna, sin cifrar (no es secreto) y visible para la IA; solo la contraseña/token va cifrada.

```
gd_credenciales
  - id
  - nombre              VARCHAR(100) UNIQUE  -- p.ej. "erp_contabilidad"
  - descripcion         MEDIUMTEXT           -- para qué sirve (esto sí lo
ve la IA)
  - usuario             VARCHAR(150)         -- opcional, sin cifrar, la IA
sí lo ve
  - valor_cifrado        VARCHAR(500)         -- la contraseña/API
key/token — Fernet, lib.cifrado del repo core
  - fecha_actualizacion TIMESTAMP
```

Decisión clave: **se gestiona solo desde el panel, nunca por conversación con la IA** — mismo motivo por el que la clave de Mistral/OpenAI ya se gestiona así hoy (Fase 3): si el cliente se la dictara a su IA por chat, quedaría escrita en el historial de esa conversación. La herramienta MCP `listar_credenciales_gd()` es de solo lectura de metadatos — nombre, descripción y usuario, nunca la contraseña. No hay (todavía) ninguna herramienta que *use* una credencial para conectarse a algo: eso es una herramienta por servicio, y sin un caso real no tenía sentido inventarla. `CredencialesCRUD.valor_descifrado()` existe para cuando llegue ese caso (uso interno del backend, nunca expuesto tal cual por una API ni una tool).

Endpoints del panel (`admin_gestion_documental_actual`, igual que las API keys — es la única información realmente sensible de este panel):

```
GET/POST   /v1/proyectos/{id}/credenciales
PUT/DELETE /v1/proyectos/{id}/credenciales/{id_credencial}
```

Tabla de estilos (2026-09-09)

Por cada tipo de informe que la IA pueda construir (p.ej. "informe_mensual", "memoria_anual"): instrucciones de cómo montarlo, más sus imágenes fijas (logo, plantilla...), subidas una vez desde el panel — un estilo puede tener varias imágenes.

```
gd_estilos
  - id
  - nombre_tipo          VARCHAR(100) UNIQUE   -- "informe_mensual"
  - instrucciones        MEDIUMTEXT            -- estructura, tono,
apartados
  - fecha_actualizacion  TIMESTAMP

gd_estilos_imagenes
  - id
  - id_estilo         FK -> gd_estilos.id, ON DELETE CASCADE
  - nombre            VARCHAR(150)   -- "logo_cabecera"
  - descripcion_uso   MEDIUMTEXT     -- cuándo/cómo usarla
  - imagen            BLOB (BYTEA en Postgres) -- el binario, directo en la
fila
  - tipo_mime         VARCHAR(50)    -- "image/png"
  - fecha_alta        TIMESTAMP
```

La imagen se guarda como `BYTEA` directo en la fila — no hay almacén de ficheros en este proyecto y montar uno (S3, disco) solo para unos pocos logos/plantillas habría sido sobredimensionado.

Cómo le llegan las imágenes a la IA (decisión 2026-09-09, con el usuario): el servidor MCP y el panel son dos backends con autenticación distinta (`x-api-key` la IA, JWT el panel), así que una URL del panel no le sirve a la IA sin más. Se descartó montar un endpoint público sin login. En su lugar, `obtener_estilo_informe_gd(nombre_tipo)` devuelve las imágenes **dentro de la propia respuesta de la tool**, como contenido de imagen nativo del protocolo MCP (`fastmcp.utilities.types.Image`) — sin URLs, sin permisos nuevos que gestionar. Cada imagen va precedida de un bloque de texto con su nombre y

su `descripcion_uso`, para que la IA sepa cuál es cuál. Probado de punta a punta: la respuesta trae 3 bloques (instrucciones, texto identificador + imagen), cada imagen con su `mime_type` correcto.

Endpoints del panel: lectura (`GET`) abierta a cualquiera con acceso al proyecto — no es información sensible, es contenido de plantilla. Alta/edición/borrado (incluida la subida de imágenes): cualquiera salvo el rol invitado (`escritor_gestion_documental_actual`, 2026-09-21; antes era solo admin, como las API keys). Regla dada por el usuario: todos los usuarios escriben en todo salvo invitado, que solo lee. Las tools MCP se rigen por el rol de la clave de API, no por una persona (2026-09-21): cada key es operador (lee y escribe) o invitado (solo lee), se elige al emitirla en el panel. Las tools que solo leen llevan `@mcp_app.tool(annotations=SOLO_LECTURA)` (`mcp_server/herramientas/_anotaciones.py`); una tool nueva sin marcar se bloquea para claves invitadas — si solo lee, márcala. Ver `docs/decisiones.md` del core → "Rol de la key".

```
GET/POST   /v1/proyectos/{id}/estilos
PUT/DELETE /v1/proyectos/{id}/estilos/{id_estilo}
GET        /v1/proyectos/{id}/estilos/{id_estilo}/imagenes
(metadatos, sin el binario)
POST       /v1/proyectos/{id}/estilos/{id_estilo}/imagenes
(multipart)
GET
/v1/proyectos/{id}/estilos/{id_estilo}/imagenes/{id_imagen}/contenido
(sirve el binario, con sesión)
DELETE     /v1/proyectos/{id}/estilos/{id_estilo}/imagenes/{id_imagen}
```

El frontend nunca pone el binario en un `<img src="...">` con el token en la URL (quedaría en logs del servidor) — lo trae con `fetch()` (cabecera `Authorization`) y lo muestra con un `object URL` (ver `ImagenEstilo` en `EstiloDetalle.jsx`).

Escritura desde la IA y documentos guía (2026-09-21): a petición del usuario, la IA también puede dar de alta y editar estilos, y vincular documentos a ellos:

- `crear_estilo_informe_gd(nombre_tipo, instrucciones)` y `actualizar_estilo_informe_gd(nombre_tipo, instrucciones)` (esta reemplaza las instrucciones completas). No hay tool para borrar un estilo (arrastraría sus imágenes) ni una imagen — eso sigue siendo solo del panel.
- `subir_imagen_estilo_gd(nombre_tipo, nombre, descripcion_uso, imagen_base64 | url)` (2026-09-21): la imagen llega por `imagen_base64` (solo viable si la IA tiene el archivo al alcance, p.ej. Claude Code — una imagen adjunta al chat no la puede leer como bytes) o por `url` pública directa. Se valida por los primeros bytes (PNG/JPEG/GIF/WEBP, máx. 5 MB), no por el tipo que declare quien la envía. La descarga por URL rechaza direcciones no públicas y no sigue redirecciones (para que no sirva de sonda contra servicios internos del servidor). Si ya existe una imagen con ese `nombre` en el estilo, se reemplaza.
- `gd_documentos.id_estilo` (INT, opcional, FK → `gd_estilos.id`, `ON DELETE SET NULL`): un documento que explica cómo elaborar un tipo de informe se vincula a su estilo con `asignar_estilo_documento_gd(id_documento, nombre_tipo)` (sin `nombre_tipo` quita el vínculo). Un documento va a un solo estilo; un estilo puede tener varios documentos guía. Se guarda el `id`, no el nombre, para que renombrar un estilo no rompa el vínculo (ver regla "id, nunca slug").

- `obtener_estilo_informe_gd` añade, tras las instrucciones, la lista de documentos guía no archivados (`id` + nombre) para que la IA los lea con `obtener_documento_completo_gd` antes de redactar; este último devuelve ahora `estilo_informe` (nombre del estilo vinculado o `null`).
- Esquema desplegado en los 5 servidores activos (2026-09-22, incluye `codemartia.multiplika.net`, dado de alta después del punto anterior — no lo tenía).
- **Exportar a PDF (2026-09-22)**: tanto el botón "PDF..." del panel (abre un modal con la opción de incluir metadatos en el pie de página, preferencia recordada en `localStorage`) como `generar_informe_pdf_gd` incrustan ahora las imágenes fijas (logo...) del estilo — el panel usa automáticamente `documento.id_estilo`, sin que haga falta indicarlo a mano. Lógica compartida en `imagenes_estilo_html()` (`database/modelos/_mod_pdf.py`). Las instrucciones de texto del estilo siguen sin aplicarse al render (solo las usa la IA para redactar el markdown de antemano).
- **Ya no pendiente**: el panel sí muestra y permite cambiar el vínculo documento-estilo (selector "Estilo de informe" en la edición de `DocumentoDetalle.jsx`, endpoint `PUT /documentos/{id}/estilo`).

Certificados de entidades (2026-09-22)

Tabla `gd_certificados` para certificados digitales (.p12/.pfx con contraseña, o .cer/.pem sin ella), calcada del patrón de `gd_credenciales` (cifrado Fernet vía `lib.cifrado` del núcleo). Esquema en `database/dependencias/_dep_certificados.py`, CRUD y extracción en `database/modelos/_mod_certificados.py`:

- Columnas: `nombre`, `descripcion`, `id_entidad` (FK nullable a `gd_entidades`, **N:1** — un certificado cuelga de como mucho una entidad, vínculo opcional, a diferencia de documentos↔entidades que es N:M), datos extraídos en claro del propio certificado (`titular`, `nif_titular`, `emisor`, `numero_serie`, `fecha_emision`, `fecha_caducidad`), y dos campos cifrados: `archivo_cifrado` (el fichero original, cifrado siempre — también para .cer aunque solo tenga la parte pública, por coherencia de código) y `contrasena_cifrada` (nullable, solo si es .p12/.pfx). Cifrado como texto (`cifrar()`/`descifrar()` de `lib/cifrado.py` operan sobre `str`, no bytes — el archivo binario va en base64 antes de cifrar).
- Extracción automática al subir (`_parsear_certificado()`): prueba primero como PKCS12 (con la contraseña dada, que puede ser None) y si falla como X.509 plano (PEM o DER) — de ahí saca `subject` (CN + `NameOID.SERIAL_NUMBER`, que en certificados españoles suele llevar el NIF), `issuer` CN, número de serie y fechas de validez. Si no se puede leer con ninguno de los dos métodos, `ValueError` → 400, no se guarda nada a medias. Probado con un certificado autofirmado real (PEM y PKCS12, contraseña correcta/incorrecta/archivo basura) antes de desplegar.
- `cryptography` añadida a `requirements.txt` de este proyecto (antes solo llegaba transitiva vía `fastmcp`→`authlib`).
- Aviso de caducidad: mismo mecanismo que documentos (`fecha_caducidad` + `aviso_caducidad_enviado`), `scripts/avisar_caducidades.py` manda ahora un único correo combinado con las dos secciones ("Documentos por caducar" / "Certificados por caducar") cuando hay algo de cualquiera de los dos tipos.
- Renovar un certificado caducado = subir uno nuevo (fila nueva), no se sobrescribe el archivo de uno existente — igual que credenciales tampoco permite cambiar el secreto ya guardado, mantiene histórico. Solo se puede editar `nombre`/`descripcion`/`id_entidad` (`PUT .../certificados/{id}`).

- Sin tools MCP (como credenciales, la IA nunca ve el secreto) — todo desde el panel: pantalla `Certificados.jsx` (listar, subir con archivo + contraseña opcional + entidad opcional, descargar original, ver el log de uso, eliminar). Endpoints en `panel_cliente/servidor.py`, todos restringidos a admin_tenant (`admin_gestion_documental_actual`, igual que credenciales).

Trazabilidad de uso: cada descarga del archivo queda registrada en `gd_certificados_log` (`id_certificado`, `certificado_nombre` denormalizado, `id_usuario`, `accion`, `detalle`, `fecha`). FK `id_certificado` en `ON DELETE SET NULL` (no CASCADE) — el registro de auditoría sobrevive aunque se borre el certificado, gracias al nombre denormalizado. Log de solo escritura: sin endpoint de editar ni borrar filas. `CertificadosLogCRUD.registrar()` se llama desde `GET .../certificados/{id}/archivo` (la única "uso" que existe hoy) con `accion="descarga"`; queda genérica para que cualquier operación futura pase por el mismo sitio. Requiere un usuario real del tenant, no un superadmin (`admin_certificados_actual` en `panel_cliente/dependencias.py`, mismo motivo que `puede_editar_documento_actual` con `gd_historial_ediciones`: el `id_usuario` referencia `mcpc_usuarios`). El panel muestra el log en un modal ("Ver uso") por certificado.

Estructura del repo: igual que cualquier proyecto de cliente de Multiplika MCP (2026-09-06)

Este proyecto no inventa una estructura propia — sigue exactamente la misma que ya usa cualquier proyecto de cliente sobre Multiplika MCP, documentada en `guia_nuevo_proyecto.md` del repo core:

```
multiplika_gestion_documental_mcp/
├── database/
│   ├── dependencias/    # esquema (gd_configuracion, gd_documentos,
gd_versiones, gd_chunks,
│   │                    #          gd_credenciales, gd_estilos,
gd_estilos_imagenes)
│   ├── modelos/         # CRUD y lógica de negocio
│   └── update_db.py     # migración de esta base (vía mtpk-postgres, no
mtpk-mariadb)
├── mcp_server/
│   └── herramientas/    # las tools MCP (consultar_documentos_gd,
buscar_documentos_gd,
│                        #  archivar/desarchivar, listar_credenciales_gd,
│                        #
listar_estilos_informe_gd/obtener_estilo_informe_gd...)
└── panel_cliente/       # panel web del cliente — ver "Panel del cliente"
en el plan de fases
    ├── servidor.py       # backend FastAPI: documentos, API keys,
credenciales, estilos
    └── frontend/         # React — se despliega ya compilado (dist/), sin
Node en el servidor
```

Repo propio e independiente del core (`multiplika_mcp`), igual que cualquier otro proyecto de cliente — se junta con la plataforma solo en el servidor, vía `sync.py`. La única diferencia respecto a un proyecto de cliente normal es el motor de base de datos (`mtpk-postgres` en vez de `mtpk-mariadb`), ya contemplado en `guia_nuevo_proyecto.md`.

Aprovisionamiento y catálogo de clientes/servidores

Esto **no es de este proyecto** — es una capacidad del core de Multiplika MCP (detección/instalación del motor de BD + saber qué clientes hay y en qué servidor), que gestión documental usa pero no posee. Ver doc [[multiplika-mcp-catalogo-clientes]].

Plan de implementación (fases, 2026-09-06)

Requisito previo, no es una fase de este proyecto: `mtpk-postgres` (librería independiente, repo propio) tiene que estar disponible sin el bug de índices — ya lo está, v0.1.1 en PyPI (ver [[mtpk-postgres-adaptacion]]).

Fase 1 — Esqueleto del repo + esquema

- Crear el repo `multiplika_gestion_documental_mcp` con la estructura estándar (`database/dependencias`, `database/modelos`, `mcp_server/herramientas`), siguiendo `guia_nuevo_proyecto.md` de `multiplika_mcp`.
- Definir `_dep_configuracion.py`, `_dep_documentos.py`, `_dep_versiones.py`, `_dep_chunks.py` con `mtpk-postgres`.
- Aplicar el esquema contra una Postgres de pruebas (`update_db.py`), incluyendo `CREATE EXTENSION vector`.

Fase 2 — Ingesta y consulta básicas (modo fulltext, sin depender aún de OpenAI) — hecho (2026-09-07)

- Modelos CRUD (`_mod_documentos.py`, `_mod_versiones.py`).
- Tool `guardar_documento_gd` (documento nuevo vs. nueva versión, con las preguntas mínimas ya definidas arriba, en una única transacción).
- Tool `buscar_documentos_gd`.
- Tool `consultar_documentos_gd` en modo `fulltext` únicamente, con una configuración de búsqueda en español que ignora acentos (`spanish_unaccent`, creada al aprovisionar la base — ver `scripts/aprovisionar_proyecto.py` del repo core).
- Probado de punta a punta contra una base Postgres real (proyecto de desarrollo interno, sin cliente todavía) y contra una réplica del árbol que produce `sync.py` del repo core.
- Con esto hay ya un piloto funcional mínimo, sin coste de IA externa.

Fase 3 — Modo semántico — hecho (2026-09-08)

- Proveedor de embeddings: **Mistral Embed** (`mistral-embed`, **1024 dimensiones — confirmado con una llamada real a la API**, no es el dato de catálogo de 1536 que traía OpenAI). La clave de API **la paga y gestiona la empresa (tenant) dueña del proyecto** (decidido 2026-09-08 — cada una la suya, no una clave global de la plataforma), desde el panel de administración del repo core (empresa → "Configuración IA"), cifrada en `mcpc_tenants`, nunca en un archivo del repo.
- Chunking por encabezados (`_mod_embeddings.py::chunkear_por_encabezados`) + generación de `gd_chunks` (`_mod_chunks.py`).
- `consultar_documentos_gd` ampliada para modo `semantico` (búsqueda por similitud de coseno con pgvector, operador `<=>`).
- Tool `activar_busqueda_semantica_gd` de backfill: cambia el modo y genera embeddings de todo lo ya guardado que sea buscable.
- `guardar_documento_gd` y `corregir_documento_gd` reindexan solos cuando el modo es semántico (ver `_reindexar_si_semantico` en `_herr_documentos.py`); las versiones que dejan de ser buscables (superadas por una nueva, sin histórico completo) se limpian solas.

- Probado de punta a punta contra Mistral real y Postgres real: activar, guardar (autoindexado), preguntar por dos aspectos distintos del mismo documento (cada uno prioriza el fragmento correcto), corregir (reindexa) y volver a preguntar (ya no aparece el dato viejo).
- **OpenAI disponible como alternativa** (2026-09-08, por si algún cliente lo prefiere — hoy todos usan Mistral por defecto): `gd_configuracion.proveedor_embedding` decide el proveedor real usado, elegible por cliente con `update_db.py --proveedor-embedding` al dar de alta su base (la dimensión del vector va fija en la columna, no se puede cambiar después sin migrar a mano). `_mod_embeddings.py` enruta a Mistral u OpenAI según ese valor; la clave de OpenAI se gestiona igual que la de Mistral, por empresa. Probado de verdad: la llamada llega a la API real de OpenAI (confirmado con un 401 de credenciales al usar una clave inválida — no es un fallo de código), y Mistral se comprobó que sigue funcionando igual tras el cambio.
- **Clave de API por empresa, no global** (2026-09-08): `database.config_db.get_mistral_api_key(alias_proyecto)` / `get_openai_api_key(alias_proyecto)` resuelven el proyecto → su tenant → la clave de ESE tenant. Probado de punta a punta contra el servidor real: guardar la clave desde "Configuración IA" de una empresa en el panel, y confirmar que `guardar_documento_gd`/`consultar_documentos_gd` la recogen sin ningún paso manual.

Fase 4 — Corrección de markdown + documento completo

- Tool `obtener_documento_completo_gd` — hecho (2026-09-07), probada de punta a punta (versión vigente, versión concreta, y los dos casos de error).
- Mecanismo de corrección (escenario B): **decidido 2026-09-08 — siempre el cliente, con su propia IA, sin revisión de nadie del equipo** (no hay capacidad para que alguien del equipo esté pendiente de corregir). Tool `corregir_documento_gd` — hecho, probada de punta a punta (corrige `markdown_actual`, conserva `markdown_original` intacto, marca `revisado_manualmente=true`, rechaza versiones inexistentes o de otro documento).

Panel del cliente (`panel_cliente/`) — hecho (2026-09-08)

- Decisión original: visualmente solo listado y búsqueda (solo lectura); guardar y corregir documentos sigue siendo exclusivamente por conversación con la IA — se valoró y se descartó subir archivos sin IA (markitdown vale para PDF digital, pero no para fotos/escaneados, que es justo el caso que peor resuelve un OCR sin IA). **Esto se quedó corto con el tiempo** — el panel ya tiene bastante más que lectura: archivar documentos, credenciales, estilos (con subida de imágenes) y usuarios, todo añadido después (ver más abajo). Lo que sigue intacto es el mecanismo de ingesta/corrección de documentos en sí: eso sigue siendo solo por conversación con la IA, nunca desde el panel.
- Reutiliza el login del panel de administración del repo core (mismo JWT, mismo usuario/contraseña de la empresa) y su sistema de roles por proyecto — no hay una segunda base de usuarios ni una segunda pantalla de login que mantener.
- Un tenant con varios proyectos de gestión documental ve un selector; con uno solo, entra directo — se resuelve por `mcpc_proyectos.tipo_producto = "gestion_documental"` (repo core), no hace falta que el cliente sepa nada del identificador del proyecto.
- Desplegado en `https://mcp-d.multiplika.net/gd`, servicio systemd propio (`multiplika_gd_panel`, puerto 8112) — primer proyecto de cliente con panel propio; `sync.py` del repo core ahora sabe sincronizar una carpeta `panel_cliente/` opcional (su frontend compilado no viaja por git, se copia aparte porque el servidor no tiene Node).

- Probado de punta a punta en producción real: login, selección de proyecto, listado, búsqueda por contenido, ver un documento completo creado de verdad a través del MCP, y que no se puede ver el proyecto de otra empresa ni sin tener rol asignado.

Cómo compilar el frontend para producción (gotcha real, 2026-09-11): `panel_cliente/frontend/vite.config.js` y `src/api.js` leen tres variables de entorno en build time — sin ellas, `npm run build` genera un `dist/` que solo funciona en local, no detrás del proxy de Apache:

```
cd panel_cliente/frontend
VITE_BASE=/gd/ VITE_API_ADMIN=/panel/v1 VITE_API_CLIENTE=/gd/v1 npm run
build
```

Sin `VITE_BASE`, Vite genera rutas de assets absolutas desde la raíz (`/assets/...`) en vez de con el prefijo `/gd/` bajo el que vive el panel — el navegador pide `/assets/index-xxx.js` a un dominio que solo tiene `/gd/assets/index-xxx.js`, Apache lo devuelve 404 (no hay `ProxyPass "/assets"`, solo `/mcp`, `/panel` y `/gd`) y la página sale en blanco. Sin `VITE_API_ADMIN`/`VITE_API_CLIENTE`, `api.js` usa sus valores por defecto de desarrollo (`http://127.0.0.1:8001` y `:8002`) — localhost del navegador de quien entra al panel, no del servidor, así que cualquier llamada a la API falla con "Failed to fetch" aunque la página cargue bien. Al subir el `dist/` compilado siempre hay que borrar lo anterior (`rsync --delete`) — los ficheros llevan hash de contenido en el nombre, así que un `dist/` a medio actualizar puede dejar un `index.html` apuntando a un asset ya borrado.

Archivar/desarchivar desde el panel — hecho (2026-09-09) Primera acción de escritura real del panel del cliente sobre documentos (hasta entonces solo lectura). Botón Archivar/Desarchivar en el detalle del documento, y una etiqueta gris "🗄 Archivado" tanto en el detalle como en el listado (que a partir de aquí incluye también los archivados, antes se excluían). Permiso: **para cualquiera con acceso al proyecto, no solo admins** — decidido así explícitamente: archivar es parte del día a día con los documentos, no una acción de administración como gestionar API keys. Probado de punta a punta contra `mcp-d.multiplika.net`.

Credenciales y tabla de estilos — hecho (2026-09-09)

- `gd_credenciales`: credenciales del cliente para servicios externos, cifradas, gestionadas solo desde el panel — ver "Credenciales" arriba. Tool `listar_credenciales_gd()`.
- `gd_estilos` / `gd_estilos_imagenes`: instrucciones e imágenes fijas por tipo de informe — ver "Tabla de estilos" arriba. Tools `listar_estilos_informe_gd()` / `obtener_estilo_informe_gd()` (más crear/actualizar/asignar a documento — ver "Escritura desde la IA y documentos guía").
- Primera vez que el panel del cliente permite escribir más allá de archivar: crear/editar estilos y credenciales, y subir imágenes (`multipart/form-data`, `python-multipart` añadido a `requirements.txt`).
- Probado de punta a punta contra `mcp-d.multiplika.net`: creación de una credencial y un estilo desde el panel (navegador real), subida de una imagen con miniatura visible, y verificación desde un cliente MCP real de que `listar_credenciales_gd()` nunca devuelve el valor y de que `obtener_estilo_informe_gd()` trae las instrucciones más la imagen como contenido nativo (bloque `image`, `mime_type` correcto). Datos de prueba limpiados al terminar.

Migración del panel del cliente a Mantine — hecho (2026-09-10) Todas las pantallas (`Login`, `ElegirProyecto`, `Documentos`, `DocumentoDetalle`, `ApiKeys`, `Credenciales`, `Estilos`, `EstiloDetalle`) pasan de CSS a mano a componentes de `@mantine/core` — motivo: dos fallos visuales seguidos (una clase CSS usada sin existir, botones sin separación) por mantener estilos propios. `index.css` queda reducido al fondo de página. Mismo cambio hecho en el panel de administración del repo core (ver `docs/decisiones.md` de ese repo). Probado con capturas de pantalla reales de las ocho pantallas.

Retoques posteriores a credenciales y estilos — hecho (2026-09-10)

- Credenciales: el campo `valor` (contraseña/API key) llevaba usuario+contraseña como JSON cuando hacía falta — se corrigió el mismo día: `usuario` es ahora su propia columna, sin cifrar (no es secreto), visible para la IA; solo la contraseña sigue cifrada y oculta. Ver "Credenciales" arriba.
- Credenciales: el navegador ofrecía autorrellenar esos dos campos con contraseñas guardadas de otro sitio (los reconocía como un login) — `autoComplete="off"`/`"new-password"` más los atributos que respetan 1Password/LastPass.
- Estilos: la caja de instrucciones (`rows={6}`/`{4}`) se quedaba corta y obligaba a hacer scroll — ahora `autosize` (crece con el contenido, hasta 30-40 líneas antes de scrollar) y con fuente monoespaciada en el detalle.

Pantalla de usuarios en el panel del cliente — hecho (2026-09-10) Antes, dar de alta un usuario de la empresa o asignarle un rol en un proyecto era cosa exclusiva de superadmin (repo core, por curl) — el cliente tenía que pedírselo al equipo. Decisión (a petición explícita del usuario): el admin_tenant de un proyecto también puede, y se le da una pantalla para hacerlo sin salir de este panel (`Usuarios.jsx`, botón "Usuarios" junto a API keys/Credenciales/Estilos, visible solo si `proyecto.es_admin`). Reutiliza tal cual los endpoints del panel de administración del repo core (mismo JWT, mismo patrón que `login`/`listarProyectosDeLaEmpresa` — ver `docs/decisiones.md` del core → "Alta de usuarios: también admin_tenant"), así que no hay lógica de permisos duplicada aquí. `ver_proyecto` (`GET /v1/proyectos/{id}`) devuelve ahora también `id_tenant`, necesario porque los usuarios son por tenant, no por proyecto. Sin edición ni borrado de usuarios en ningún sitio — a propósito.

Probado con una usuaria real (no el enlace de soporte de superadmin): login normal con su propio email/contraseña, ve el botón, crea otro usuario ella sola sin que nadie del equipo intervenga. Datos de prueba limpiados al terminar.

Conservar el archivo original — planificado (2026-09-10), pendiente de implementar

Hoy solo se guarda el markdown que genera la IA; la foto o el PDF que sube el cliente se descarta. Riesgo real para un cliente tipo gestoría: si la IA lee mal algo, o si hace falta el escaneado original para algo legal, no está en ningún sitio. Diseño acordado con el usuario, sin implementar todavía:

- **Nueva tabla `gd_versiones_originales`** (no columnas dentro de `gd_versiones`, para no arrastrar binarios pesados en cada búsqueda/listado normal): `id`, `id_version` (FK → `gd_versiones.id`, única — una por versión, ON DELETE CASCADE), `archivo` (BLOB/BYTEA), `tipo_mime`, `nombre_fichero` (opcional), `fecha_alta`. No todas las versiones tendrán fila aquí — una corrección (`corregir_documento_gd`) no sube archivo nuevo.
- **`guardar_documento_gd` gana tres parámetros opcionales**: `archivo_original_base64`, `archivo_tipo_mime`, `archivo_nombre`. Opcionales para no romper casos sin archivo real, pero la propia herramienta debe instruir a la IA para que los mande siempre que tenga el archivo.

- **Vía de subida decidida**: la IA manda el archivo junto con el markdown, en la misma llamada — no una subida aparte desde el panel. Motivo: un solo paso, no cambia el flujo actual de conversación. Riesgo aceptado y pendiente de probar: un PDF escaneado de muchas páginas podría no caber bien en una llamada — validar con un caso real grande antes de darlo por cerrado.
- **Base64 solo de paso, nunca almacenado así**: es la única forma de que el archivo viaje dentro de una llamada a herramienta (JSON no lleva binario), pero en cuanto llega se decodifica y se guarda como binario de verdad (BYTEA) — un archivo en base64 ocupa un 33% más que en binario, y con miles de documentos acumulándose se nota.
- **Recuperación**: botón "Descargar original" en el detalle del documento del panel, mismo patrón que las imágenes de estilos (`GET .../documentos/{id}/original/contenido`, autenticado, sirve el binario con su `tipo_mime`). Exponerlo también a la IA (para que pueda mostrarlo/adjuntarlo) queda como posible fase 2, no imprescindible para la primera versión.

Fase 5 — Piloto con un cliente real

- Elegir el primer cliente, probar el ciclo completo (subir, consultar, corregir) y ajustar según lo que salga.
- Pendiente: modelo de coste al cliente por el uso de embeddings — decisión de negocio, no técnica.

Cinco mejoras planificadas (2026-09-10), pendientes de implementar

Diseño acordado con el usuario para las cinco ideas que quedaron solo anotadas. Se implementan en este orden, de más a menos autocontenida.

1. Filtros en la búsqueda del panel — implementado y probado (2026-09-10)

`GET /v1/proyectos/{id}/documentos` acepta ahora parámetros opcionales `tipo_documento` y `fecha_desde`/`fecha_hasta` (no a la búsqueda por contenido, que sigue siendo solo el texto — son dos modos que no se combinan), con un `Select` de tipo y dos selectores de fecha en `Documentos.jsx`.

A mitad de esta implementación el cliente pidió que la lista de tipos dejara de ser cerrada — ver "Tipos de documento" más arriba, que se implementó primero por ser un prerrequisito: el `Select` de este filtro ahora carga sus opciones desde `listar_tipos_documento_gd()`/`GET /v1/proyectos/{id}/tipos-documento` en vez de una lista fija en el frontend.

2. Documentos pendientes: la pieza compartida por subida masiva e ingesta por email — implementado y probado (2026-09-10)

Decidido con el usuario: cuando un documento llega **sin nadie delante** (un lote subido al panel, o un email reenviado), no puede pasar por las preguntas que la IA siempre hace (tipo, nombre, si es una versión nueva...) — así que se queda **pendiente** hasta que el cliente habla con su IA, que lo procesa entonces con las mismas preguntas de siempre. No cambia la regla de "la IA siempre pregunta", solo cambia cuándo pregunta.

```
gd_documentos_pendientes
  - id
  - archivo          BLOB (BYTEA)   -- el fichero tal cual llegó
  - tipo_mime         VARCHAR(50)
  - nombre_original   VARCHAR(255)   -- nombre de fichero o asunto del
email
```

```
  - origen            ENUM('panel', 'email')
  - fecha_alta         TIMESTAMP
```

Tools MCP:

- `listar_documentos_pendientes_gd()` — metadatos de lo que hay en la cola (nombre, origen, fecha), para que la IA pueda avisar al cliente ("tienes 2 documentos nuevos, ¿los proceso?") al empezar una conversación.
- `obtener_documento_pendiente_gd(id)` — devuelve el fichero como contenido de imagen/archivo nativo de la respuesta (mismo mecanismo que las imágenes de estilos: `Image` para `image/*`, `File` para el resto — PDF incluido), para que la IA pueda leerlo de verdad antes de preguntar.
- `guardar_documento_gd` gana un parámetro opcional `id_documento_pendiente` — si se indica, al guardar con éxito borra esa fila de la cola.

Subida masiva (vía panel): `GET/POST /v1/proyectos/{id}/documentos-pendientes` (POST multipart, campo repetido `archivos`, una fila por fichero) y `DELETE /v1/proyectos/{id}/documentos-pendientes/{id_pendiente}` para descartar uno sin procesarlo. Abierto a cualquier usuario del proyecto (no solo admin) — mismo criterio que archivar/desarchivar: es día a día con documentos, no configuración sensible. `Documentos.jsx` tiene un botón "Subir documentos" (selector múltiple) y, si hay algo en cola, una tabla pequeña con botón "Descartar" por fila.

Ingesta por email: el proceso que lee el correo entrante inserta en esta misma tabla, con `origen='email'` (ver "3. Ingesta por email").

3. Ingesta por email — implementado (2026-09-10, rediseñado el mismo día)

Diseño descartado primero, y por qué: la idea inicial era un buzón/dominio compartido de la plataforma (`<algo-por-cliente>@documentos.multiplika.net`), con `getmail` entregando a un script vía Postfix. Al ir a montarlo, la tabla `mail_domain` de ISPConfig estaba vacía — no hay ningún dominio de correo dado de alta en este servidor — así que hacía falta crear uno nuevo, tocando DNS del dominio raíz. Se probó un atajo (una dirección bajo el propio hostname del servidor, `facturae.multiplika.net`, que Postfix ya acepta sin tocar DNS, con un alias en `/etc/aliases` que canaliza el correo directo a un script) y **el usuario lo frenó a tiempo**: no tiene sentido depender de una dirección del servidor cuando cada cliente de Gestión Documental ya tiene su propia cuenta de correo de empresa. Diseño final, mucho más simple: **cada cliente da de alta SU PROPIA cuenta de correo** (la que ya usa) en el panel, y esa misma cuenta sirve para las dos cosas — mandar los avisos de caducidad y recibir documentos.

```
gd_configuracion (añadido a la fila única)
  - email_cuenta_direccion         VARCHAR(255)
  - email_cuenta_usuario           VARCHAR(255)  -- login SMTP/IMAP si es
distinto de la dirección
  - email_cuenta_password_cifrada  VARCHAR(500)  -- Fernet, lib.cifrado,
igual que gd_credenciales
  - email_cuenta_smtp_host / _smtp_port (587) / _smtp_seguridad
ENUM('starttls','ssl','ninguna')
```

```
  - email_cuenta_imap_host / _imap_port (993) / _imap_seguridad
ENUM('ssl','starttls','ninguna')
```

`_seguridad` es explícito, elegido por el cliente en el panel — la primera versión lo adivinaba por el puerto (465 = SSL, cualquier otro = STARTTLS, e IMAP siempre SSL), lo que no vale para configuraciones fuera de lo habitual. El usuario lo pidió al revisar la pantalla.

- Panel: `GET/PUT /v1/proyectos/{id}/configuracion/cuenta-email` (solo admin) — pantalla "Cuenta de correo" (botón junto a los demás en `Documentos.jsx`). El `GET` nunca devuelve la contraseña, aunque esté guardada; el `PUT` solo la sobrescribe si se manda una nueva (dejarla vacía no la borra) — mismo patrón que `gd_credenciales`.
- `scripts/revisar_correo_entrante.py` (cron cada 15 min, `mcp_gd_revisar_correo`): por cada proyecto con cuenta configurada, se conecta por IMAP, busca correos no leídos, extrae los adjuntos válidos (imagen o PDF) de cada uno, los inserta en `gd_documentos_pendientes` con `origen='email'`, y marca el correo como leído (tenga o no adjuntos válidos, para no revisarlo otra vez). Una cuenta mal configurada (contraseña caducada, servidor caído...) de un cliente no impide revisar la de los demás — mismo `try/except` por proyecto que ya lleva `avisar_caducidades.py`.
- `scripts/avisar_caducidades.py` (ver "4." más abajo) se actualizó para mandar los avisos desde esta misma cuenta por SMTP, en vez de desde un remitente compartido del servidor.

Probado: el parseo MIME/extracción de adjuntos (con un correo sintético) y el CRUD del panel — de punta a punta con una cuenta IMAP/SMTP real todavía no, porque hace falta una cuenta de verdad para probarlo (una cuenta de prueba del propio usuario, o una que cree para el caso).

Bug real encontrado al probar el CRUD del panel (2026-09-10): `actualizar_cuenta_email` solo protegía la contraseña de un `PUT` parcial — el resto de campos (dirección, usuario, hosts, puertos) se escribían literalmente, así que corregir un solo campo (p.ej. el host SMTP) borraba todo lo demás ya guardado a `NULL`. Arreglado igual que la contraseña: solo se actualizan los campos que llegan con un valor (no `None`). Verificado por revisión de código y por la lógica ya probada del mismo patrón en la contraseña; no se ha vuelto a lanzar el test de extremo a extremo tras el arreglo.

4. Fechas de caducidad y avisos proactivos por email — implementado (2026-09-10)

- `gd_versiones.fecha_caducidad` (DATE, opcional) — junto a `fecha_documento`, no en `gd_documentos`: la caducidad es del contenido de esa versión concreta (un contrato renovado es, en la práctica, otra fecha). `gd_versiones.aviso_caducidad_enviado` (BOOLEAN) acompaña al campo: **un solo aviso por documento, no un recordatorio diario** (decidido explícitamente por el usuario al planificar esto) — se marca a `true` en cuanto se manda el correo, para no repetirlo.
- `guardar_documento_gd` gana un parámetro opcional `fecha_caducidad`.
- Tool `listar_documentos_por_caducar_gd(dias=30)` — para que la IA pueda responder si se le pregunta (independiente del aviso proactivo).
- **Aviso proactivo por email**: `gd_configuracion.email_avisos` (a quién avisar; NULL desactiva el aviso), gestionable desde el panel (`GET/PUT /v1/proyectos/{id}/configuracion/avisos`, solo admin — pantalla "Avisos de caducidad", botón junto a los demás en `Documentos.jsx`) sin depender de nosotros. Script diario `scripts/avisar_caducidades.py` (cron `mcp_gd_avisos_caducidad`, 07:15 — mismo patrón que `backup_diario.py` del core: recorre `mcpc_proyectos` con `motor='postgres'` y `tipo_producto='gestion_documental'`) que, por

cada proyecto con algo por caducar y `email_avisos` configurado, manda UN correo con todo lo pendiente y marca esas versiones como avisadas — **por SMTP desde la cuenta de correo propia del cliente** (`gd_configuracion.email_cuenta_*`, ver "3. Ingesta por email" — mismo mecanismo, se comparte la misma cuenta), sin remitente compartido de la plataforma. Sin cuenta configurada, o sin `email_avisos`, se omite ese proyecto.

Bug real encontrado al probarlo end-to-end (2026-09-10): el envío por `smtplib` no tenía manejo de errores — un `email_avisos` mal configurado (buzón inexistente, dominio caído...) de UN solo cliente propagaba la excepción hasta abortar el script entero, dejando sin avisar a TODOS los demás clientes esa noche (sin pérdida de datos: al no marcarse `aviso_caducidad_enviado`, se reintenta solo con la siguiente ejecución diaria, pero silenciosamente hasta entonces). Arreglado con un `try/except` alrededor del envío en `_procesar_proyecto()`, que solo salta ese proyecto y sigue con el resto. Autocontenido a propósito (SQL en línea contra `gd_versiones`/`gd_documentos`/`gd_configuracion`, sin pasar por `database/modelos` de este repo) para no depender de que sus imports relativos funcionen fuera del árbol de despliegue del servidor MCP — ver el docstring del propio script para el razonamiento completo. Desplegado junto a `panel_cliente/`, en `apps/mcp_server/paneles_cliente/multiplika_gestion_documental_mcp/scripts/`.

5. Exportar un informe a PDF — implementado (2026-09-10)

Sencillo para empezar (texto formateado + logo, no maquetación compleja). Tool `generar_informe_pdf_gd(markdown, nombre_archivo, nombre_tipo_estilo=None)`: convierte el markdown que acaba de escribir la IA a HTML (biblioteca `markdown`) y de ahí a PDF (`WeasyPrint` — valorado `pandoc`, no está instalado en el servidor y añadiría una dependencia externa al margen de Python), incrustando las imágenes fijas del estilo indicado (si se pasa uno) al principio del documento, y devuelve el PDF como contenido de archivo nativo de la respuesta (la IA se lo entrega al cliente directamente, sin pasar por el panel).

Infraestructura de servidor que hizo falta (no solo pip): WeasyPrint necesita bibliotecas nativas del sistema (Pango, Cairo, GDK-Pixbuf, GObject) — `libgdk-pixbuf-2.0-0` no estaba instalada en `mcp-d`; el mirror `security.debian.org` tenía el índice caducado para la versión exacta esperada, así que se instaló la versión equivalente disponible en el mirror normal (`apt-get install libgdk-pixbuf-2.0-0=2.42.2+dfsg-1+deb11u2 ...`).

Gotcha real de conda encontrado al probarlo: el proceso del servidor MCP corre en un entorno conda (`py313mcp`) **compartido por las herramientas de todos los proyectos**. Ese entorno trae su propio `libffi`, incompatible con el `libgobject` del sistema que WeasyPrint carga por `dlopen` (fallo real: `undefined symbol: ffi_type_uint32, version LIBFFI_BASE_7.0`). Instalar Pango/Cairo/GObject vía `conda install -c conda-forge pango` habría sido la solución "de manual", pero el servidor solo tiene ~1 GB de RAM libre y sin swap — el solver de conda-forge se quedó sin memoria y el proceso murió (`Terminado (killed)`), en un servidor que aloja más servicios además de este. Se descartó por riesgo de tumbar algo más mientras resuelve.

Solución adoptada, sin tocar el entorno compartido: `generar_informe_pdf_gd` lanza el render de WeasyPrint en un **subproceso aparte** (`asyncio.create_subprocess_exec`, HTML por stdin, PDF por stdout), con `LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libffi.so.7` puesto solo en el entorno de ese subproceso — fuerza a usar el libffi del sistema (compatible con el gobject del sistema) sin cambiar nada del proceso principal del servidor ni de las demás herramientas que comparten ese mismo entorno conda.

Orden de implementación: 1 → 2 (con subida masiva) → 5 → 4 → 3. Las cinco, implementadas y desplegadas el 2026-09-10 — la 3 tuvo que rediseñarse a mitad de camino (ver su apartado: se descartó un buzón/dominio propio de la plataforma a favor de que cada cliente use su propia cuenta de correo).

Normativas legales como caso de uso (2026-09-11)

Un cliente puede usar la Gestión Documental para consultar normativas legales (leyes, reglamentos, órdenes) además de sus propios documentos. No hizo falta esquema nuevo salvo dos campos:

- **Tipo de documento**: `"normativa"` como cualquier otro valor de `tipo_documento` — el catálogo ya es texto libre (ver "Tipos de documento" más arriba), no hace falta ningún cambio.
- **Vigente / derogada**: ya cubierto por el mecanismo de versiones existente. Cuando una normativa se sustituye por otra, es una nueva versión del mismo documento (`guardar_documento_gd` con `id_documento_existente`): la anterior deja de ser `vigente` pero queda en el histórico, tal cual ya funciona hoy para cualquier documento corregido.
- **`gd_versiones.url_origen`** (VARCHAR(1000), opcional, campo nuevo): URL pública de donde se leyó esa versión (p.ej. la ficha del BOE de esa publicación concreta). Es solo referencia para poder volver a consultar la fuente si hace falta — el markdown guardado sigue siendo la fuente de verdad, no se depende de que la URL siga viva. Cada versión de una normativa suele tener su propia URL oficial (una revisión posterior es una publicación distinta), así que vive en `gd_versiones`, no en `gd_documentos`.
- **`gd_documentos.ambito`** (VARCHAR(50), opcional, campo nuevo, 2026-09-11): ámbito de aplicación — p.ej. "nacional", "autonómica", "privada" para normativas. Texto libre como `tipo_documento`, pero sin catálogo de sugerencias (a diferencia de `gd_tipos_documento`): son pocos valores esperados y no aplica a todos los tipos de documento (una factura no tiene ámbito). Vive en `gd_documentos`, no en `gd_versiones` — no cambia entre versiones del mismo documento. Solo se fija al crear el documento (parámetro `ambito` de `guardar_documento_gd`); no se puede cambiar al subir una versión nueva de uno ya existente, igual que `tipo_documento`. Decidido explícitamente para NO meterlo dentro de `tipo_documento` (p.ej. "normativa nacional", "normativa autonómica") porque el filtro de tipo del panel dejaría de poder agrupar todas las normativas juntas.

Flujo de ingesta desde una URL: no hay una tool de descarga en el servidor MCP — es el propio cliente IA (Claude Desktop/Code, con su capacidad de leer URLs) quien descarga y lee el PDF, igual que hoy lee un PDF adjunto en la conversación. `guardar_documento_gd` gana el parámetro opcional `url_origen` para guardar esa referencia; el resto del pipeline (preguntas de siempre, markdown, indexado) no cambia.

Implementado y desplegado (2026-09-11): columnas en `_dep_versiones.py`/`_dep_documentos.py`, parámetros en `guardar_documento()` (`database/modelos/_mod_versiones.py`) y en la tool `guardar_documento_gd` (`mcp_server/herramientas/_herr_documentos.py`), devueltos también por `obtener_documento_completo_gd` y por el detalle de documento del panel (`GET /v1/proyectos/{id}/documentos/{id}`, mostrado en `DocumentoDetalle.jsx` como filas "Fuente" y "Ámbito" de la tabla de metadatos). Migración (`update_db.py`) aplicada en la base del proyecto de pruebas interno (`desarrollo_interno_gestion_documental_dev`) — sigue pendiente para cualquier otra base de cliente que se provisione antes de tener esto ya en su `update_db.py` de referencia (hoy no hay ninguna: solo existe ese proyecto de pruebas).

Ayuda para el cliente: mismo texto en la IA y en el panel (2026-09-11)

El cliente puede preguntarle a su IA "¿cómo funciona esto?" y también quiere un sitio equivalente en el panel — un único texto explicando todo el flujo (guardar, cola de pendientes, buscar, corregir, versiones, archivar, caducidades, normativas/ámbito, informes PDF, qué se gestiona desde el panel), sin mantener dos redacciones distintas.

Dónde vive el contenido: `database/ayuda_gd.py` (constante `TEXTO_AYUDA_GD`, markdown en texto plano) — no en `docs/` porque `sync.py` no despliega `docs/` a ningún sitio (ver "Cómo se organizan los proyectos" del repo core), y `database/` sí: es la única carpeta que `sincroniza_proyectos()` copia **entera** tanto a `apps/mcp_server/proyectos/p<id>/database/` (para las tools MCP) como a `apps/mcp_server/paneles_cliente/<repo>/database/` (para el panel) — ver "Cómo compilar el frontend..." más arriba sobre esa doble copia. Poniendo el contenido ahí, con el mismo import relativo `..database.ayuda_gd` que ya usan `modelos`/`dependencias`, un solo archivo de origen llega a los dos árboles de despliegue sin duplicar texto ni depender de rutas de fichero calculadas a mano (que además serían distintas en cada árbol).

- Tool MCP **`obtener_ayuda_gd()`** (`mcp_server/herramientas/_herr_documentos.py`): devuelve el texto tal cual, para que la IA lo use en conversación.
- Panel: **`GET /v1/ayuda`** (`panel_cliente/servidor.py`) — sin autenticación ni proyecto, es contenido estático igual para cualquier cliente. Pantalla nueva "Ayuda" (`Ayuda.jsx`), enlazada con un botón más en `Documentos.jsx`/`App.jsx`, mismo patrón de texto monoespaciado que ya usa `DocumentoDetalle.jsx` para el markdown de un documento (no hace falta una librería de renderizado de markdown para esto).

Formularios MCP (elicitation) para datos que da el cliente (2026-09-15)

Hasta ahora, los datos que solo puede rellenar el cliente (nunca deducirlos del contenido) se le pedían en texto suelto por chat, uno detrás de otro. El protocolo MCP tiene un mecanismo pensado justo para esto — **elicitation**: la propia tool le pide a la aplicación (Claude Desktop/Code, ChatGPT en modo desarrollador...) que le muestre un formulario de verdad al cliente; la respuesta vuelve a la tool ya estructurada, sin que la IA tenga que transcribir ni interpretar nada. `fastmcp` (framework ya usado por el core) lo trae de serie: `ctx: Context` como parámetro de la tool (FastMCP lo inyecta solo, no aparece en el esquema que ve la IA) y `await ctx.elicit(mensaje, response_type=UnModeloPydantic)`.

Dos tools migradas a este mecanismo:

- **`guardar_documento_gd`**: los campos que "se preguntan siempre al cliente" (nombre, tipo de documento, fecha del documento, fecha de caducidad, ámbito, si el histórico debe quedar buscable) salen de la firma de la tool y se piden con un formulario (`_DatosDocumento`) en cuanto se llama. Lo que sigue siendo parámetro de la tool es lo que decide la propia IA a partir de la conversación o del contenido: `markdown`, `id_documento_existente` (tras confirmar con `buscar_documentos_gd`), `id_documento_pendiente`, `url_origen`, `borrador`, `incumplimientos`. El mensaje del formulario incluye los tipos de documento ya usados en el proyecto (consultando `gd_tipos_documento`), a modo de sugerencia.
- **`crear_credencial_gd`** (tool nueva): antes solo se podía dar de alta una credencial desde el panel, precisamente para que su valor no quedara escrito en la conversación con la IA (ver "Credenciales" más arriba). Con elicitation esa razón deja de obligar a pasar por el panel: el formulario (`_DatosCredencialNueva`) recoge nombre/descripción/valor/usuario directamente del cliente, y el valor va del formulario al servidor sin que la IA lo vea ni lo escriba en ningún mensaje — igual de

seguro que el panel, pero sin salir de la conversación. Editar o borrar una credencial ya creada sigue siendo solo desde el panel (no hay tool para eso).

Si el cliente cierra el formulario sin rellenarlo (`action != "accept"`), la tool falla con `ToolError` y la propia IA debe avisar de que no se ha guardado nada, sin insistir ni pedir los datos por chat como alternativa.

Sin probar todavía contra un cliente MCP real (ni Claude ni ChatGPT) — depende de que la aplicación que conecta soporte elicitation; en ChatGPT hace falta además su "Developer Mode". Antes de dar esto por bueno hay que verificarlo con una conexión real.

Registros de seguimiento: comentarios, anotaciones, observaciones, incidencias (2026-09-16)

El cliente quiere poder dejar comentarios/anotaciones/observaciones sobre un documento, abrir incidencias con un responsable asignado, resolverlas dejando constancia de cómo, relacionar estos registros entre sí, y que todo eso siga disponible aunque el documento tenga una versión nueva.

Decisión clave — una sola tabla (`gd_registros`), no cinco. Un campo `tipo` (`comentario` / `anotacion` / `observacion` / `incidencia` / `resolucion`) distingue cada caso. Ventaja principal: relacionar dos registros entre sí es la misma mecánica sea cual sea su tipo, y una resolución no deja de ser más que un registro de tipo `resolucion` enlazado a su incidencia — no hace falta un mecanismo aparte para "resolver".

- **Cuelga de `gd_documentos`, no de `gd_versiones`** — a propósito, para que un registro siga apareciendo entero con cualquier versión nueva del documento sin copiarlo ni migrarlo nada. Lleva además `id_version_origen` (nullable, informativo): en qué versión estaba el documento cuando se creó, pero nunca decide en qué versiones se ve.
- **`creado_por` es texto libre, no un id de usuario real.** Desde el panel se podría resolver de sobra (hay sesión, `puede_editar_documento_actual` ya da el `id_usuario`, ver "Edición desde el panel"), pero desde una tool MCP no hay una identidad de persona fiable — la API key identifica al proyecto, no a quién escribe en ese momento del chat. En vez de dos mecanismos distintos según el origen, un único campo de texto: el panel lo rellena solo con el nombre real de quien tiene la sesión, y por chat se le pregunta a la IA que lo pida si no lo sabe ya (nunca que lo invente).
- **`responsable` es texto libre en cualquier tipo de registro, no solo en incidencias** — decisión explícita del cliente: no hace falta que sea un usuario real del sistema, y no vale la pena limitarlo a un tipo concreto.
- **`estado` (`abierta`/`resuelta`) solo tiene sentido en incidencias** — estructuralmente disponible en cualquier registro (columna nullable en la misma tabla, no una tabla aparte) por simplicidad, pero NULL fuera de ese tipo.
- **Relaciones en tabla aparte (`gd_registros_relaciones`)**, no una columna `id_registro_padre`: un registro puede relacionarse con más de uno, y en cualquier dirección. `tipo_relacion` es una etiqueta libre (p.ej. "relacionado", "duplica"); `"resuelve"` la pone siempre `resolver_incidencia()`, nunca a mano — así una incidencia resuelta tiene garantizado tener su resolución enlazada (misma transacción: crear el registro de resolución, crear la relación, marcar `estado='resuelta'`).
- **Crear una `resolucion` directamente está bloqueado** (`crear_registro()` solo acepta los otros cuatro tipos) — la única vía es `resolver_incidencia_gd`/el endpoint de resolver, para que ese invariante (incidencia resuelta ⇒ resolución enlazada) no se pueda romper por otro camino.

Tools MCP (`mcp_server/herramientas/_herr_registros.py`): `crear_registro_documento_gd`, `listar_registros_documento_gd`, `resolver_incidencia_gd`,

`asignar_responsable_registro_gd`, `relacionar_registros_gd`. Sin elicitation aquí — a diferencia de `guardar_documento_gd`/`crear_credencial_gd`, `creado_por`/`responsable` son campos sueltos de bajo riesgo (no secretos, no un cluster de varios campos), así que van como parámetro normal de la tool con instrucción en el docstring de preguntarlos por chat en vez de inventarlos.

Panel: sección nueva "Comentarios e incidencias" en `DocumentoDetalle.jsx` (`componentes/RegistrosDocumento.jsx`) — listar, crear, resolver, asignar responsable y relacionar, todo desde la ficha del documento. Mismo criterio de permisos que editar el documento (`puede_editar_documento_actual`): invitado puede leer, no crear.

Estado (2026-09-16): implementado y desplegado en los 4 servidores (`mcp-d`, `maderascilpe`, `mgsasesores`, `adimelsl`), esquema aplicado sobre Postgres real en las 4 bases sin tocar datos existentes (verificado contando filas de `gd_documentos`/`gd_versiones` antes y después). Sin probar todavía con un cliente MCP real end-to-end (Claude Desktop/Code) ni desde el panel en producción — pendiente de una prueba real antes de darlo por cerrado del todo.

Quién decidió: usuario (confirmado en conversación, incluida la decisión explícita de que `responsable` sea libre en todos los tipos). Reversible: sí — son dos tablas nuevas, sin tocar ninguna existente; quitar las tools/el endpoint/la sección del panel no afecta a nada más.

Aviso de incidencias abiertas por email (2026-09-17)

Mismo problema que resolvió "Fechas de caducidad y avisos proactivos por email" (más arriba), aplicado a incidencias: si nadie entra al panel ni le pregunta a la IA, abrir una incidencia y asignarle un responsable no avisa a nadie de que existe.

Decisión: un aviso por asignación, no un recordatorio diario mientras siga abierta. Se valoró mandar un resumen diario con todo lo abierto (como hace `avisar_caducidades.py` cada día hasta que caduca), pero una incidencia puede tardar semanas en resolverse — recibir el mismo aviso cada día durante ese tiempo es ruido, no ayuda a que se resuelva antes. En vez de eso: columna `gd_registros.aviso_responsable_enviado` (mismo patrón que `gd_versiones.aviso_caducidad_enviado`), y `RegistrosCRUD.actualizar_responsable()` la reinicia a `false` cada vez que cambia el responsable — así reasignarla sí vuelve a avisar (a quien corresponda ahora), pero dejarla como está no genera más correos.

El correo va a `email_avisos`, no al responsable directamente — `responsable` es texto libre (ver decisión de arriba), no necesariamente un email real ni un usuario del sistema, así que no hay a quién escribirle de forma fiable. El cuerpo del correo menciona el nombre del responsable en cada línea para que quien lo reciba (normalmente un admin, la misma persona que ya recibe los avisos de caducidad) sepa a quién avisar.

`scripts/avisar_incidencias.py` — autocontenido, mismo patrón exacto que `avisar_caducidades.py` (SQL en línea, no importa `database/modelos`, recorre `mcpc_proyectos` con `motor='postgres'` y `tipo_producto='gestion_documental'`, manda desde la cuenta de correo propia del cliente, un fallo de un proyecto no tumba a los demás). Cron `mcp_gd_avisos_incidencias`, 07:20 (justo después del de caducidad, 07:15, para no competir por la misma conexión SMTP del cliente al mismo tiempo).

Quién decidió: usuario pidió "programar esos avisos" tras una recomendación del asistente; el diseño concreto (una vez por asignación, destinatario `email_avisos`) lo decidió el asistente por las razones de arriba — a revisar si no encaja con lo que el cliente esperaba. Reversible: sí — una columna nueva (`aviso_responsable_enviado`, `NOT NULL DEFAULT false`, no rompe nada existente) y un script/cron independiente; quitar el cron basta para desactivarlo del todo.

El responsable puede llevar su propio email (2026-09-17)

A los pocos minutos de cerrar lo de arriba, pregunta obvia en retrospectiva: si `responsable` es texto libre, ¿por qué no dejar apuntar también su email, para escribirle a él en vez de al `email_avisos` genérico?

`gd_registros.responsable_email`, columna nueva, opcional — no se valida el formato (es un dato de contacto, no una cuenta del sistema; ver la misma filosofía de `responsable` en sí). `RegistrosCRUD.actualizar_responsable()` la actualiza junto con `responsable` y sigue reiniciando `aviso_responsable_enviado` si cualquiera de los dos cambia (así añadir el email que faltaba también dispara un aviso nuevo, no solo cambiar de persona).

`avisar_incidencias.py` ahora separa las incidencias pendientes de aviso en dos grupos: las que tienen `responsable_email` van agrupadas por dirección (una persona con varias incidencias recibe un solo correo con todas) directo a esa persona; las que no, siguen cayendo en el correo a `email_avisos` de siempre, mencionando el nombre. Un proyecto sin `email_avisos` configurado ya no bloquea los avisos a responsables que sí tienen email — cada grupo se manda con independencia del otro.

Tools/API: `crear_registro_documento_gd`, `asignar_responsable_registro_gd` y sus endpoints/formulario del panel llevan `responsable_email` opcional junto a `responsable`. Mismo criterio que el nombre: si viene por chat, la IA lo pasa solo si el cliente lo da, nunca lo inventa.

Quién decidió: usuario. Reversible: sí — columna nullable, no rompe nada existente; sin `responsable_email` en ningún registro, el comportamiento es idéntico al de antes de este cambio.

Grupos de documentos (2026-09-17)

El cliente quiere poder agrupar documentos "para un fin concreto" (una auditoría, un expediente) sin que eso implique que todo documento tenga que estar en un grupo.

Decisión: muchos-a-muchos, no un `id_grupo` en `gd_documentos` — confirmado con el usuario: un mismo documento puede estar en varios grupos a la vez (p.ej. una factura relevante tanto para "Auditoría 2026" como para "Proyecto X"). De ahí `gd_documentos_grupos`, tabla de relación pura, mismo patrón que `gd_registros_relaciones` — índice único `(id_documento, id_grupo)` para que asignar dos veces no duplique fila, `ON DELETE CASCADE` en ambos lados (borrar un documento o un grupo limpia sus relaciones solas).

`gd_grupos` deliberadamente mínima: solo `nombre` (único) y `proposito` — el usuario descartó explícitamente añadir responsable/estado al grupo en esta primera versión; se puede ampliar después si hace falta, sin migrar nada existente.

Borrar un grupo no toca los documentos — solo desaparecen sus filas de `gd_documentos_grupos` (cascada), los documentos en sí quedan intactos. Sin "archivar" (a diferencia de documentos/versiones): un grupo es metadato organizativo sin historial que conservar, borrar sin más es suficiente.

Tools MCP (`mcp_server/herramientas/_herr_grupos.py`): `crear_grupo_gd`, `listar_grupos_gd`, `asignar_documento_grupo_gd`, `quitar_documento_grupo_gd`, `listar_documentos_grupo_gd`, `eliminar_grupo_gd`.

Panel: pantalla nueva "Grupos" (`Grupos.jsx` → lista y crea; `GrupoDetalle.jsx` → edita, borra, añade/quita documentos), enlazada en la barra lateral igual que Tipos de documento/Estilos (no gated a admin — es día a día, no configuración). En la ficha de cada documento, `componentes/GruposDocumento.jsx` muestra sus grupos como chips (con su propósito en el tooltip) y deja añadir/quitar sin salir de ahí. Mismo criterio de permisos que registros: cualquiera con acceso lee, `puede_editar` para crear/asignar/quitar/borrar.

Ajuste (2026-09-17, mismo día): crear un grupo nuevo también desde la ficha del documento. La primera versión de `GruposDocumento.jsx` solo dejaba asignar a grupos ya existentes — para crear uno había que ir a la pantalla "Grupos" y volver. El usuario no quería salir del documento en ningún caso: se añadió un formulario compacto (nombre + propósito) plegable en el propio widget, que crea el grupo y lo asigna al documento en el mismo paso (dos llamadas seguidas, `crearGrupo` + `asignarDocumentoGrupo`, sin transacción — aceptable: si la segunda fallase, el grupo queda creado pero sin asignar, visible y corregible a mano desde "Grupos", no es un estado inconsistente grave). **Mismo ajuste en `EntidadesDocumento.jsx` (2026-09-17, pedido después)** — con una diferencia: en vez de un formulario compacto inline (la entidad tiene 11 campos, no 2), se abre en un `Modal` reutilizando `FormularioEntidad.jsx` (el mismo componente que ya usan `Entidades.jsx`/`EntidadDetalle.jsx`) — sigue sin salir de la ficha del documento, solo que como superposición en vez de un bloque plegable, por el volumen de campos. Mismo patrón de dos llamadas seguidas (`crearEntidad` + `relacionarDocumentoEntidad`). De paso, los dos selectores de "añadir/relacionar" (grupos y entidades) se agrandaron (de `size="xs" w=200/220` a `size="sm" w=340`) — ya eran buscables (`searchable`), pero al ser tan pequeños no se notaba.

Fuera de alcance por ahora: filtrar el listado principal de documentos por grupo (se puede añadir después, es solo una columna más en `DocumentosCRUD.buscar()` y un `Select` más en `Documentos.jsx`, pero no se pidió y no hacía falta para lo de asignar).

Quién decidió: usuario (nombre del feature, y las dos preguntas de diseño — muchos-a-muchos, y grupo mínimo sin responsable/estado). Reversible: sí — dos tablas nuevas, sin tocar ninguna existente.

Entidades (2026-09-17)

El cliente quiere una tabla de clientes/proveedores/empleados relacionable con documentos, con solo los campos básicos de contacto.

`gd_entidades`: `nombre`, `tipo`, `direccion`, `localidad`, `provincia`, `codigo_postal`, `pais`, `telefono`, `email`, `observaciones`, `codigo` (para vincular con el ERP del cliente, único si se rellena). `tipo` es el campo que el cliente llamó "entidad" en la petición original ("id, nombre, entidad, dirección...") — aclarado en conversación: es el qué-es-esto (cliente/proveedor/empleado), no un segundo nombre. Se renombró a `tipo` para no repetir "entidad" dentro de la propia tabla `gd_entidades` y para seguir el mismo patrón que `gd_documentos.tipo_documento` — catálogo abierto, texto libre, sin tabla de catálogo aparte (el cliente pidió expresamente "solo los campos básicos").

Relación con documentos: muchos-a-muchos, mismo patrón que grupos — `gd_documentos_entidades`, sin campo de "rol" en la relación (a diferencia de `gd_registros_relaciones`, aquí no se pidió distinguir

"cliente" de "proveedor" en una factura con dos entidades; si hiciera falta más adelante, se añade un `rol` a esta tabla sin tocar el resto).

`codigo_postal` es texto, no número (hay formatos con letras fuera de España, y `pais` ya es un campo aparte). Sin "archivar" en entidades — a diferencia de documentos: no se pidió, y borrar sin más ya dijo el cliente que no debía tocar los documentos relacionados (limpieza en cascada solo de la fila de relación, `ON DELETE CASCADE` en `gd_documentos_entidades`, nunca en `gd_documentos` ni en `gd_entidades` entre sí).

Tools MCP (`mcp_server/herramientas/_herr_entidades.py`): `crear_entidad_gd`, `listar_entidades_gd` (por nombre y/o tipo), `actualizar_entidad_gd` (solo toca los campos que se pasen, no borra los que se omiten), `eliminar_entidad_gd`, `relacionar_documento_entidad_gd`, `quitar_relacion_documento_entidad_gd`, `listar_documentos_entidad_gd`, `listar_entidades_documento_gd`.

Panel: pantalla "Entidades" (lista + buscador por nombre/tipo + modal de alta) y `EntidadDetalle.jsx` (editar todos los campos, borrar, relacionar/quitar documentos) — mismo componente de formulario (`FormularioEntidad.jsx`) en los dos sitios, para no mantener 11 campos duplicados. En la ficha de cada documento, `EntidadesDocumento.jsx` (mismo patrón que `GruposDocumento.jsx`) muestra sus entidades como chips y deja añadir/quitar sin salir de ahí. `tipo` se rellena con un `Autocomplete` (texto libre con sugerencias de los tipos ya usados), no un `Select` cerrado.

Quién decidió: usuario (los 11 campos, la relación muchos-a-muchos con documentos); el nombre de columna `tipo` (en vez de repetir `entidad`) lo decidió el asistente por claridad, ver arriba. Reversible: sí — dos tablas nuevas, sin tocar ninguna existente.

Comunicaciones y envíos (2026-09-17)

El despacho manda comunicaciones a clientes y otras entidades con frecuencia, cada una con contenido distinto (nombre, importe, fecha...). Se pidió: el documento hace de plantilla con variables dentro del texto, y desde ahí se programan envíos por email a las entidades que se elijan, a mano o en automático.

Decisión: la plantilla es un documento normal, no una tabla aparte — cualquier `gd_documentos` puede usarse para esto, sin marcarlo de ninguna forma especial. Las variables son `{{asi}}` (regex `\{\{\s*(\w+)\s*\}\}`) dentro de `nombre` (asunto) y del markdown (cuerpo); no hace falta declararlas en ningún sitio aparte, se detectan solas leyendo el texto. Se aprovecha todo lo que ya existe para documentos (versiones, comentarios/incidencias, grupos, entidades relacionadas) sin construir nada nuevo para "plantillas".

Tres decisiones de producto, confirmadas con el usuario antes de construir esto:

1. **Variables compartidas por lote, no una tabla por entidad.** Al mandar el mismo documento a varias entidades a la vez, las variables que NO coinciden con un campo de la entidad (ver abajo) se escriben una sola vez y valen para todas — no hay una cuadrícula de "un valor por entidad y variable". Más simple, cubre circulares y avisos generales; no vale para "cada cliente debe un importe distinto" en el mismo lote (eso necesitaría una entrada de datos por fila, descartado por complejidad para esta primera versión).
2. **Solo envíos programados para una fecha/hora concreta, no recurrentes.** Programar "cada mes" habría exigido que las variables se recalculen solas en cada repetición (fechas relativas, importes que

cambian) — no encaja con variables de valor fijo escritas una vez. Para repetir, se crea un envío nuevo cada vez.

3. **El correo se manda con formato (HTML), no texto plano** — a diferencia de los avisos de caducidad/incidencias (que son avisos internos), una comunicación a un cliente cuida más la imagen. Se reaprovecha la misma conversión que ya usa `generar_informe_pdf_gd` (librería `markdown`, ya en `requirements.txt`) — el correo va como `multipart/alternative`, con una parte de texto plano (el markdown sin convertir, para clientes que no muestran HTML) y una parte HTML.

El contenido se resuelve y se guarda YA HECHO al crear el envío, no al mandarlo (`gd_envios.asunto`/`contenido_html`/`contenido_texto`) — así un envío programado manda exactamente lo que se aprobó, aunque el documento se edite después; y el cron que manda los programados (`scripts/procesar_envios.py`) no necesita la librería `markdown` ni la lógica de variables, solo lee esas columnas y las manda tal cual — mismo patrón autocontenido que `avisar_caducidades.py`/`avisar_incidencias.py`, y más simple que esos dos porque no hay nada que calcular en el momento de mandar.

Variables "auto" vs "personalizadas: auto = coincide con una columna de gd_entidades (nombre, tipo, direccion, localidad, provincia, codigo_postal, pais, telefono, email, observaciones, codigo) y se resuelve sola por entidad; cualquier otra (importe, fecha_limite...) es personalizada. Una variable sin valor se deja tal cual en el texto ({{asi}}` literal) en vez de desaparecer en blanco — para que se note en la vista previa que falta rellenarla.

`gd_envios`: un registro por entidad destinataria (aunque el "lote" cree varios a la vez) — cada uno necesita su propio `email_destino` y sus propias variables auto. `estado` (`programado`/`enviado`/`fallido`/`cancelado`); `fecha_programada` NULL significa "ya" — se manda en el mismo paso de crear el envío, sin esperar al cron. Sin email en la entidad → se crea igual, en `fallido` con el motivo, en vez de omitirse en silencio.

Tools MCP (`mcp_server/herramientas/_herr_envios.py`): `detectar_variables_documento_gd`, `crear_envios_gd` (crea el lote; si no se programa, el propio flujo de chat llama después a `enviar_envio_gd` por cada uno), `enviar_envio_gd`, `listar_envios_documento_gd`, `listar_envios_entidad_gd`, `cancelar_envio_gd`, `reintentar_envio_gd`.

Panel: pestaña nueva "Enviar" en `DocumentoDetalle.jsx` (`componentes/EnviosDocumento.jsx`) — detecta variables, elige entidades (`MultiSelect`), pide las personalizadas, fecha opcional, vista previa (con sustitución hecha en el propio JS, reutilizando `Markdown.jsx` — no hace falta ida y vuelta al servidor solo para previsualizar) y la lista de envíos de ese documento con cancelar/reintentar.

Cron `mcp_gd_procesar_envios`, cada 5 minutos (`scripts/procesar_envios.py`) — más frecuente que los avisos diarios porque aquí la hora programada importa de verdad, no es un resumen del día.

Quién decidió: usuario (la idea completa, y las tres decisiones de producto de arriba, cada una con dos opciones planteadas por el asistente). Reversible: sí — una tabla nueva, sin tocar ninguna existente; quitar el cron desactiva la parte automática sin afectar a los envíos manuales.

Editor y visor de documentos, y exportar a PDF (2026-09-17)

Tres retoques seguidos sobre cómo se ve/edita un documento en el panel, a petición del usuario ("un poco cutre"):

- **Editor**: el `Textarea` monoespaciado de "Editar" se sustituyó por `MDEditor` (`@uiw/react-md-editor`) — mismo componente que ya usaba "Contexto de negocio", así que no fue una dependencia nueva. Barra de formato + vista previa en vivo (`preview="live"`), más alto (560px) que en Contexto por tratarse de documentos completos, no dos párrafos.
- **Visor** (`Markdown.jsx`): fallo real, no solo de estilo — `react-markdown` sin `remark-gfm` no interpreta tablas (ni tachado, ni listas de tareas): una tabla se veía como una línea de texto con barras verticales sueltas. El CSS de `.contenido-documento` para tablas ya existía desde antes pero nunca se llegaba a aplicar porque no se generaba ningún `<table>`. Añadido `remark-gfm` (ya lo trae `react-markdown` como dependencia opcional habitual, se instaló aparte) y completado el CSS que faltaba (h4-h6, blockquote, código/pre, hr).
- **Exportar a PDF**: botón "Descargar PDF" en la ficha del documento — la versión vigente, con el mismo formato que se ve en pantalla. Reutiliza el mecanismo que ya existía para los informes que redacta la IA (`generar_informe_pdf_gd`: markdown → HTML → WeasyPrint en subproceso aparte, por el conflicto de `libffi` con el entorno conda compartido, ver comentario en el código). Ese render se sacó de `mcp_server/herramientas/_herr_pdf.py` a `database/modelos/_mod_pdf.py` — la única carpeta que llega a los dos árboles de despliegue (proyecto y panel) — para que el panel pudiera reutilizarlo sin duplicar el subproceso ni su gotcha de libffi; `_herr_pdf.py` quedó como una fachada fina sobre ese módulo. CSS de impresión aparte (`CSS_DOCUMENTO`, distinto de `CSS_INFORME`): en pantalla el cuerpo va en gris "dimmed" de baja intensidad, que en papel impreso se lee mal, así que el PDF usa texto oscuro/negro sólido.

Scripts reutilizables (2026-09-25)

Tabla `gd_scripts` para que la IA guarde los scripts que ya ha escrito para tareas puntuales del cliente (un backup, un informe ad-hoc, una migración...) y los reaproveche en vez de reescribirlos cada vez. Esquema en `database/dependencias/_dep_scripts.py`, CRUD en `database/modelos/_mod_scripts.py`:

- Columnas: `titulo`, `descripcion` (qué hace y cuándo usarlo — lo que se lee al buscar), `lenguaje` (texto libre, catálogo abierto como `gd_entidades.tipo`), `codigo`, `etiquetas` (texto libre, opcional), `creado_por`, `veces_usado`/`fecha_ultimo_uso` (para ver cuáles compensan tener guardados) y `archivado` (igual que `gd_documentos.archivado`, para retirar uno obsoleto sin borrarlo).
- Tools MCP (`mcp_server/herramientas/_herr_scripts.py`): `crear_script_gd`, `buscar_scripts_gd`/`obtener_script_gd` (solo lectura), `actualizar_script_gd`, `marcar_uso_script_gd` (incrementa el contador — la IA la llama al reaprovechar uno, no al crearlo) y `archivar_script_gd`/`desarchivar_script_gd`.
- Panel: pantalla "Scripts" (listar/buscar por título, descripción o etiqueta, y el detalle) — solo para consultar y curar metadatos, **el código no se puede editar desde el panel a propósito** (`ActualizarScript` en `panel_cliente/servidor.py` no lleva ese campo): quien corrige o mejora un script es la IA, vía `actualizar_script_gd`. El código se muestra coloreado según el lenguaje con `@mantine/code-highlight` (adaptador `highlight.js`, añadidos como dependencias nuevas del frontend) — tema oscuro fijo (`highlight.js/styles/github-dark.css`), no ligado al claro/oscuro del panel, para no tener que mantener dos hojas de estilo sincronizadas con el selector de tema.

- **Relación con documentos (2026-09-25)**: tabla `gd_documentos_scripts`, muchos-a-muchos, mismo patrón que `gd_documentos_entidades` (índice único `(id_documento, id_script)`, `ON DELETE CASCADE` en ambos sentidos — a diferencia de `gd_scripts` en solitario, esta tabla sí tiene FK hacia `gd_documentos`, así que un despliegue en un servidor real tiene que registrar también `TABLA_DOCUMENTOS` en el script `ManagerDB` ad-hoc, ver [[despliegue_panel_cliente_gotchas]]). Tools MCP: `relacionar_documento_script_gd`, `quitar_relacion_documento_script_gd`, `listar_scripts_documento_gd`/`listar_documentos_script_gd` (solo lectura). Panel: `ScriptsDocumento.jsx` (fila de badges en `DocumentoDetalle.jsx`, mismo patrón visual que `EntidadesDocumento.jsx` pero sin modal de "crear y relacionar" — los scripts no se crean a mano) y la sección "Documentos relacionados" en `ScriptDetalle.jsx` (idéntica a la de `EntidadDetalle.jsx`).

Quién decidió: usuario (los tres retoques); la extracción a `_mod_pdf.py` y el CSS de impresión aparte los decidió el asistente por las razones de arriba. Reversible: sí — ninguno toca el esquema; quitar el botón o revertir el import no afecta a nada más.

GDs enlazadas: una central de conocimiento consultada por otras (2026-09-27, plan — sin implementar)

Idea del usuario: una gestión documental "central" alberga una gran base de conocimiento (p.ej. toda la normativa de la asesoría) y otras GDs "enlazadas" (una por cliente) la consultan para responder lo que no está en sus propios documentos. La central se mantiene una sola vez; las enlazadas se benefician al instante.

Tres requisitos fijados por el usuario: la enlazada **solo lee**; la central puede **reservar** cierta información para que no salga hacia fuera; y la central **toma nota de las preguntas que no supo resolver** para cubrirlas después.

Cómo se comunican. La enlazada llama al servidor MCP público de la central por HTTPS (`https://<dominio central>/mcp`) como un cliente más, con una API key emitida por la central — exactamente el mismo mecanismo que ya usa el chat del panel contra su propio MCP (`panel_cliente/chat_ia.py`, cliente `fastmcp`). No hay acceso directo entre bases de datos (son VPS distintos) ni código nuevo de red: se reutiliza `/mcp` tal cual. La búsqueda (fulltext o semántica, con las claves de embedding de la central) la hace la central; la enlazada no necesita nada de eso.

Rol nuevo en el núcleo: `gd_enlazada` (id 5, `mcpc_roles`). Una key con este rol es la que la central le da a cada enlazada. Se comporta como `invitado` (lista blanca `readOnlyHint`, `apps/mcp_server/core/solo_lectura.py`) **más** las herramientas etiquetadas con `tags={"gd_enlazada"}` (etiqueta nativa de `fastmcp`, sin depender de campos extra en las anotaciones MCP, que no admiten extras — comprobado en fastmcp 4.0.3). Hace falta un rol propio, no vale `invitado`, por dos motivos: la central tiene que saber que quien pregunta es una GD ajena (para ocultarle lo reservado y para anotar la pregunta), y necesita permitirle una única escritura (anotar). El contexto de la petición (`auth.py`, `_proyecto_actual`) expone también el `nombre` de la key — la central lo usa como "quién preguntó" (el admin la nombra al emitirla: "GD Maderas Cilpe"). Selector de rol en ambos paneles: opción nueva "GD enlazada". El rol se propaga solo al desplegar (`insertar_registros_iniciales` del ORM inserta los que falten).

Lado central (repo GD):

- `gd_documentos.reservado` BOOLEAN, default False: "no se comparte con GDs enlazadas". Filtro `AND d.reservado = false` en `buscar_fulltext`, `buscar_semantico`, `buscar` (metadatos) y `obtener_documento_completo_gd` **solo cuando la key es de rol `gd_enlazada`** — los usuarios de la propia central lo ven todo. Toggle "Reservado" en `DocumentoDetalle.jsx` (junto a archivar), badge en el listado, y parámetro opcional en `guardar_documento_gd`/tool `marcar_reservado_gd`.
    - Por defecto se comparte todo y se marca la excepción (no al revés): una central existe para compartir; obligar a marcar uno a uno los cientos de documentos compartibles sería lo tedioso. Ejemplo: de 300 normativas, se reservan las 5 notas internas de la asesoría.
- Tabla nueva `gd_preguntas_pendientes`: `pregunta`, `origen` (nombre de la key = qué GD preguntó), `motivo` ENUM(`sin_resultados`, `no_resuelta`), `veces` (misma pregunta normalizada + mismo origen → se incrementa, no se duplica), `estado` ENUM(`pendiente`, `resuelta`), `fecha_primera`, `fecha_ultima`. Sin FKs.
- Dos vías de anotación: **automática** — `consultar_documentos_gd` llamada por una key `gd_enlazada` que devuelve 0 fragmentos anota `sin_resultados` sola (no depende de que la IA se acuerde); y **explícita** — tool `anotar_pregunta_pendiente_gd(pregunta, detalle)` etiquetada `gd_enlazada`, para cuando hubo resultados pero no respondían (`no_resuelta`), que es lo que solo la IA del lado enlazado puede juzgar.
- Panel de la central: pantalla "Preguntas pendientes" (solo admin): listado ordenado por `veces` desc, filtro por estado/origen, botón "marcar resuelta". Tools de solo lectura `listar_preguntas_pendientes_gd` + `resolver_pregunta_pendiente_gd` para hacerlo también desde el chat.

Lado enlazado (mismo repo — toda GD puede ser central y enlazada a la vez):

- Tabla `gd_enlaces`: `nombre`, `url` (el `/mcp` de la central), `api_key_cifrada` (Fernet, `lib.cifrado` del núcleo, igual que `gd_credenciales`), `activo`, `fecha_alta`. Varias filas: enlazar con más de una central cuesta lo mismo que con una.
- Panel: pantalla "GDs enlazadas" (solo admin): alta (nombre, URL, key), botón "probar conexión" (hace `list_tools` contra la central), activar/desactivar, borrar. La key nunca se vuelve a mostrar.
- Tools MCP nuevas (`_herr_enlaces.py`), todas `SOLO_LECTURA` porque no tocan datos propios: `consultar_gds_enlazadas_gd(pregunta, limite)` — recorre los enlaces activos, llama a `consultar_documentos_gd` de cada central y devuelve los fragmentos con un campo `origen`; `obtener_documento_enlazado_gd(id_enlace, id_documento)` — el documento completo de la central; `anotar_pregunta_pendiente_enlazada_gd(id_enlace, pregunta, detalle)` — reenvía la anotación a la central. Esta última también va marcada de solo lectura a propósito: no modifica nada de la enlazada y no queremos que el chat pida confirmación por cada anotación (sería un estorbo y las preguntas ya viajan a la central al consultarla). Una central caída no rompe la respuesta: se devuelve el error de ese enlace como un resultado más y se sigue con el resto.
- `SYSTEM_PROMPT` de `chat_ia.py`: instrucción de consultar primero los documentos propios, después las GDs enlazadas si existen, citar de qué GD viene cada fragmento, y anotar en la central lo que no se resolvió. Sin coste de IA en la central: allí solo se ejecuta una búsqueda; el gasto de Claude sigue siendo únicamente el del chat de la enlazada (una llamada de herramienta más por turno cuando aplica).

Fases: (1) núcleo: rol 5, middleware con etiquetas, nombre de key en el contexto, selectores de rol; (2) central: `reservado` + filtros + `gd_preguntas_pendientes` + tools + panel; (3) enlazada: `gd_enlaces` +

tools + panel + prompt; (4) prueba real en mcp-d contra sí misma (enlace apuntando a su propio dominio público — vale como prueba de punta a punta) y después contra un servidor real; (5) despliegue a los 5 servidores (todos llevan el mismo código; cada uno hace de central o de enlazada según lo que configure).

Quién decidió: usuario (idea y los tres requisitos); el asistente propuso rol propio en vez de reutilizar `invitado`, "compartido por defecto", la anotación automática al no haber resultados, y no pedir confirmación para anotar. Reversible: sí — una columna con default y dos tablas nuevas sin FKs; borrar los enlaces desactiva todo sin tocar documentos.